Міністерство освіти і науки України

Національний технічний університет України

«Київський політехнічний інститут імені Ігоря

Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра обчислювальної техніки

ЗВІТ

з Лабораторної роботи №4

«Аналітичні SQL-запити (OLAP)»

з дисципліни

“Бази даних”

Виконали: студенти групи ІМ-55
Горбань А.О

Долгіх Д.М

Перевірив: Хмельницький А.А

Київ - 2026

Тема: Аналітичні SQL-запити (OLAP)

Мета: Освоїти практичні навички маніпулювання даними та складання аналітичних OLAP-запитів у СУБД PostgreSQL (pgAdmin). Навчитися застосовувати агрегатні функції (COUNT, SUM, AVG, MIN, MAX), оператори групування (GROUP BY), фільтрацію згрупованих результатів (HAVING), різноманітні типи з'єднань таблиць (INNER JOIN, LEFT JOIN, CROSS JOIN) та вкладені підзапити (Subqueries).

2. КОД

```
SELECT worker_role, COUNT(*) AS total_workers FROM worker GROUP BY worker_role;
```

```
SELECT payment_method, SUM(price) AS revenue FROM payment GROUP BY payment_method;
```

```
SELECT c.category_name, COUNT(p.product_id) AS total_products FROM category c

JOIN product p ON c.category_id = p.category_id GROUP BY c.category_name HAVING COUNT(p.product_id) > 0;
```

```
SELECT p.product_name, p.price FROM product p

LEFT JOIN order_item oi ON p.product_id = oi.product_id WHERE oi.order_item_id IS NULL;
```

```
SELECT c.category_name, ROUND(AVG(p.price), 2) AS average_price FROM product p

JOIN category c ON p.category_id = c.category_id GROUP BY c.category_name;
```

```
SELECT MIN(order_date) AS first_order_date, MAX(order_date) AS last_order_date FROM orders;
```

```
SELECT c.client_name, c.last_name, o.order_id, o.status FROM client c

LEFT JOIN orders o ON c.client_id = o.client_id;
```

```
SELECT product_name, price FROM product WHERE price > (SELECT AVG(price) FROM product);
```

```
SELECT client_name, last_name, (SELECT COUNT(*) FROM orders WHERE
orders.client_id = client.client_id) AS orders_count FROM client;
```

```
SELECT product_name FROM product WHERE category_id IN (SELECT category_id
FROM category WHERE category_name = 'Electronics');
```

```
SELECT w.last_name AS worker_name, c.client_name FROM worker w CROSS JOIN client
c;
```

```
SELECT COUNT(*) AS total_orders FROM orders;
```

```
SELECT country, COUNT(*) AS addresses_count FROM address GROUP BY country;
```

```
SELECT COUNT(*) AS order_with_product, SUM(quantity) AS product_sold FROM
order_item WHERE product_id = 1;
```

```
SELECT MAX(order_price) AS max_order_price FROM orders;
```

```
SELECT status, COUNT(*) AS orders_count FROM orders GROUP BY status;
```

```
SELECT client_id, ROUND(AVG(order_price), 2) AS avg_spent FROM orders GROUP BY
client_id;
```

```
SELECT payment_status, COUNT(*) AS payments_count FROM payment WHERE price >
1000 GROUP BY payment_status;
```

```
SELECT client_id, COUNT(*) AS total_orders, SUM(order_price) AS total_spent FROM
orders GROUP BY client_id;
```

```
SELECT o.order_date, o.order_price, c.client_name || ' ' || c.last_name AS full_name, c.email,
c.phone_number, p.product_name

FROM orders o

INNER JOIN client c ON c.client_id = o.client_id

INNER JOIN order_item oi ON oi.order_id = o.order_id

INNER JOIN product p ON p.product_id = oi.product_id;


SELECT o.order_date, o.order_price, a.country, a.city, a.street, p.product_name

FROM orders o

LEFT JOIN address a ON o.address_id = a.address_id

LEFT JOIN order_item oi ON oi.order_id = o.order_id

LEFT JOIN product p ON p.product_id = oi.product_id;


SELECT c.client_id, c.client_name, SUM(o.order_price) AS total_spent

FROM client c LEFT JOIN orders o ON c.client_id = o.client_id

GROUP BY c.client_id, c.client_name HAVING SUM(o.order_price) > 5000;


SELECT a.city, COUNT(o.order_id) AS total_orders

FROM orders o LEFT JOIN address a ON o.address_id = a.address_id

GROUP BY a.city HAVING COUNT(o.order_id) >= 2;


SELECT c.client_name, c.last_name, o.order_price FROM client c

INNER JOIN orders o ON c.client_id = o.client_id

WHERE o.order_price > (SELECT AVG(order_price) FROM orders);
```

```
SELECT * FROM client WHERE client_id IN (SELECT client_id FROM orders WHERE
status = 'confirmed');



SELECT c.client_name, c.last_name, a.city FROM client c

LEFT JOIN address a ON c.client_id = a.client_id

WHERE a.city IN (SELECT city FROM address WHERE client_id = 1) AND c.client_id != 1;



SELECT city, avg_per_person FROM (

    SELECT a.city, SUM(o.order_price)/COUNT(o.order_id) AS avg_per_person

    FROM address a LEFT JOIN orders o ON a.address_id = o.address_id GROUP BY a.city

) AS subquery WHERE avg_per_person > 1000;
```

3. ХІД ВИКОНАННЯ РОБОТИ ТА ОПИС SQL-ЗАПИТІВ

Частина 1. Агрегаційні функції, групування (GROUP BY) та фільтрація (HAVING)

Запит 1. Кількість працівників за ролями

Опис: Обчислює загальну кількість співробітників у кожній із посадових ролей (адміністратор, менеджер, оператор, кур'єр).

```
SELECT worker_role, COUNT(*) AS total_workers

FROM worker

GROUP BY worker_role;
```

Запит 2. Загальна виручка за кожним методом оплати

Опис: Підраховує сумарний дохід від здійснених платежів, розділений за способом проведення транзакцій.

```
SELECT payment_method, SUM(price) AS revenue

FROM payment

GROUP BY payment_method;
```

Запит 3. Кількість продуктів у кожній категорії

Опис: Об'єднує товари з відповідними категоріями та відфільтровує за допомогою

HAVING лише ті категорії, де є принаймні один товар.

```
SELECT c.category_name, COUNT(p.product_id) AS total_products

FROM category c

JOIN product p ON c.category_id = p.category_id

GROUP BY c.category_name

HAVING COUNT(p.product_id) > 0;
```

Запит 5. Середня ціна продуктів у кожній категорії

Опис: Розраховує середню вартість одиниці товару для кожної категорії з округленням до двох знаків після коми.

```
SELECT c.category_name, ROUND(AVG(p.price), 2) AS average_price

FROM product p

JOIN category c ON p.category_id = c.category_id

GROUP BY c.category_name;
```

Запит 6. Дата першого та останнього замовлення

Опис: Визначає найранішу та найновішу дати реєстрації замовлень у системі за допомогою функцій MIN та MAX.

```
SELECT MIN(order_date) AS first_order_date, MAX(order_date) AS last_order_date FROM orders;
```

Запит 12. Загальна кількість замовлень

Опис: Обчислює загальну кількість укладених угодах у базі даних.

```
SELECT COUNT(*) AS total_orders

FROM orders;
```

Запит 13. Кількість адрес по кожній країні

Опис: Групує адреси за країнами реєстрації клієнтів та рахує їх кількість.

```
SELECT country, COUNT(*) AS addresses_count

FROM address

GROUP BY country;
```

Запит 14. Продажі для товару з ID = 1

Опис: Визначає кількість замовлень, де фігурує товар №1, та загальну кількість проданих одиниць (SUM(quantity)).

```
SELECT COUNT(*) AS order_with_product, SUM(quantity) AS product_sold

FROM order_item

WHERE product_id = 1;
```

Запит 15. Найдорожче замовлення

Опис: Знаходить максимальну суму чека серед усіх замовлень магазину.

```
SELECT MAX(order_price) AS max_order_price

FROM orders;
```

Запит 16. Кількість замовлень у кожному статусі

Опис: Аналізує розподіл замовлень за їхнім поточним станом виконання (confirmed, processing, shipped, delivered).

```
SELECT status, COUNT(*) AS orders_count

FROM orders

GROUP BY status;
```

Запит 17. Середня сума витрачених коштів на замовлення для клієнта

Опис: Обчислює середній чек покупок окремо для кожного клієнта.

```
SELECT client_id, ROUND(AVG(order_price), 2) AS avg_spent

FROM orders

GROUP BY client_id;
```

Запит 18. Кількість платежів на суму понад 1000 грн

Опис: Підраховує кількість проведених та непроведених платежів із сумою транзакції понад 1000 грн.

```
SELECT payment_status, COUNT(*) AS payments_count

FROM payment

WHERE price > 1000
```

GROUP BY payment_status;

Запит 19. Загальні витрати кожного клієнта

Опис: Підраховує сумарну вартість усіх покупок і кількість зроблених замовлень для кожного ідентифікатора клієнта.

SELECT client_id, COUNT(*) AS total_orders, SUM(order_price) AS total_spent

FROM orders

GROUP BY client_id;

Запит 22. Клієнти, які витратили понад 5000 грн

Опис: За допомогою HAVING відбирає лише активних покупців із загальною сумою покупок понад 5000 грн.

SELECT c.client_id, c.client_name, SUM(o.order_price) AS total_spent

FROM client c

LEFT JOIN orders o ON c.client_id = o.client_id

GROUP BY c.client_id, c.client_name

HAVING SUM(o.order_price) > 5000;

Запит 23. Міста, в які зроблено не менше 2 замовлень

Опис: Визначає найпопулярніші міста доставки на основі кількості оформлених угод.

SELECT a.city, COUNT(o.order_id) AS total_orders

FROM orders o

LEFT JOIN address a ON o.address_id = a.address_id

GROUP BY a.city

HAVING COUNT(o.order_id) >= 2;

Частина 2. З'єднання таблиць (JOINs)

Запит 4. Продукти, які ніколи не замовлялись (LEFT JOIN)

Опис: За допомогою LEFT JOIN та перевірки IS NULL знаходить товари з каталогу, які ще жодного разу не купувалися.

SELECT p.product_name, p.price

FROM product p

LEFT JOIN order_item oi ON p.product_id = oi.product_id

WHERE oi.order_item_id IS NULL;

Запит 7. Список клієнтів та їхніх замовлень (LEFT JOIN)

Опис: Виводить повний перелік покупців, включаючи тих, у кого ще немає створених замовлень.

SELECT c.client_name, c.last_name, o.order_id, o.status

FROM client c

LEFT JOIN orders o ON c.client_id = o.client_id;

Запит 11. Усі можливі пари працівник-клієнт (CROSS JOIN)

Опис: Генерує декартів добуток таблиць працівників та клієнтів для виведення всіх можливих комбінацій.

SELECT w.last_name AS worker_name, c.client_name

FROM worker w

CROSS JOIN client c;

Запит 20. Деталізація клієнтів та замовлених ними товарів (INNER JOIN)

Опис: Об'єднує 4 таблиці (orders, client, order_item, product) та формує повне ім'я клієнта за допомогою конкатенації ||.

SELECT o.order_date, o.order_price, c.client_name || ' ' || c.last_name AS full_name, c.email, c.phone_number, p.product_name

FROM orders o

INNER JOIN client c ON c.client_id = o.client_id

INNER JOIN order_item oi ON oi.order_id = o.order_id

INNER JOIN product p ON p.product_id = oi.product_id;

Запит 21. Замовлення, адреси та назви продуктів (LEFT JOIN)

Опис: Зводить інформацію про дату укладання угоди, геолокацію доставки та назву придбаного товару.

SELECT o.order_date, o.order_price, a.country, a.city, a.street, p.product_name

FROM orders o

LEFT JOIN address a ON o.address_id = a.address_id

LEFT JOIN order_item oi ON oi.order_id = o.order_id

LEFT JOIN product p ON p.product_id = oi.product_id;

Частина 3. Вкладені підзапити (Subqueries)

Запит 8. Продукти, ціна яких вища за середню (Підзапит у WHERE)

Опис: Зрівнює ціну кожного товару з обчислюваним скалярним під запитом середнім значенням по всьому каталогу.

SELECT product_name, price

FROM product

WHERE price > (SELECT AVG(price) FROM product);

Запит 9. Кількість замовлень на кожного клієнта (Підзапит у SELECT)

Опис: Обчислює обсяг покупок окремою корельованою колонкою у вибірці клієнтів.

SELECT client_name, last_name,

(SELECT COUNT(*) FROM orders WHERE orders.client_id = client.client_id) AS orders_count

FROM client;

Запит 10. Товари з категорії 'Electronics' (Підзапит з IN)

Опис: Знаходить товари, ідентифікатор категорії яких відповідає текстовій назві Electronics у підзапити.

SELECT product_name

FROM product WHERE category_id IN (SELECT category_id

FROM category WHERE category_name = 'Electronics');

Запит 24. Клієнти із замовленнями, вищими за середній чек (JOIN + Підзапит)

Опис: Виводить покупців, чиї чеки перевищують середню вартість угод по магазину.

SELECT c.client_name, c.last_name, o.order_price

FROM client c

```
INNER JOIN orders o ON c.client_id = o.client_id

WHERE o.order_price > (SELECT AVG(order_price) FROM orders);
```

Запит 25. Клієнти із підтвердженими замовленнями (Підзапит з IN)

Опис: Вибирає профільні дані користувачів, які мають замовлення у статусі confirmed.

```
SELECT *

FROM client

WHERE client_id IN (SELECT client_id FROM orders WHERE status = 'confirmed');
```

Запит 26. Клієнти з того ж міста, що й клієнт №1 (Підзапит + JOIN)

Опис: Шукає інших покупців, які проживають у тому ж місті, що й перший клієнт.

```
SELECT c.client_name, c.last_name, a.city

FROM client c

LEFT JOIN address a ON c.client_id = a.client_id

WHERE a.city IN (SELECT city FROM address WHERE client_id = 1)

    AND c.client_id != 1;
```

Запит 27. Міста із середнім чеком > 1000 грн (Підзапит у FROM)

Опис: Формує тимчасову віртуальну таблицю у блоці FROM та фільтрує середні показники витрат по містах.

```
SELECT city, avg_per_person

FROM (

        SELECT a.city, SUM(o.order_price)/COUNT(o.order_id) AS avg_per_person

        FROM address a

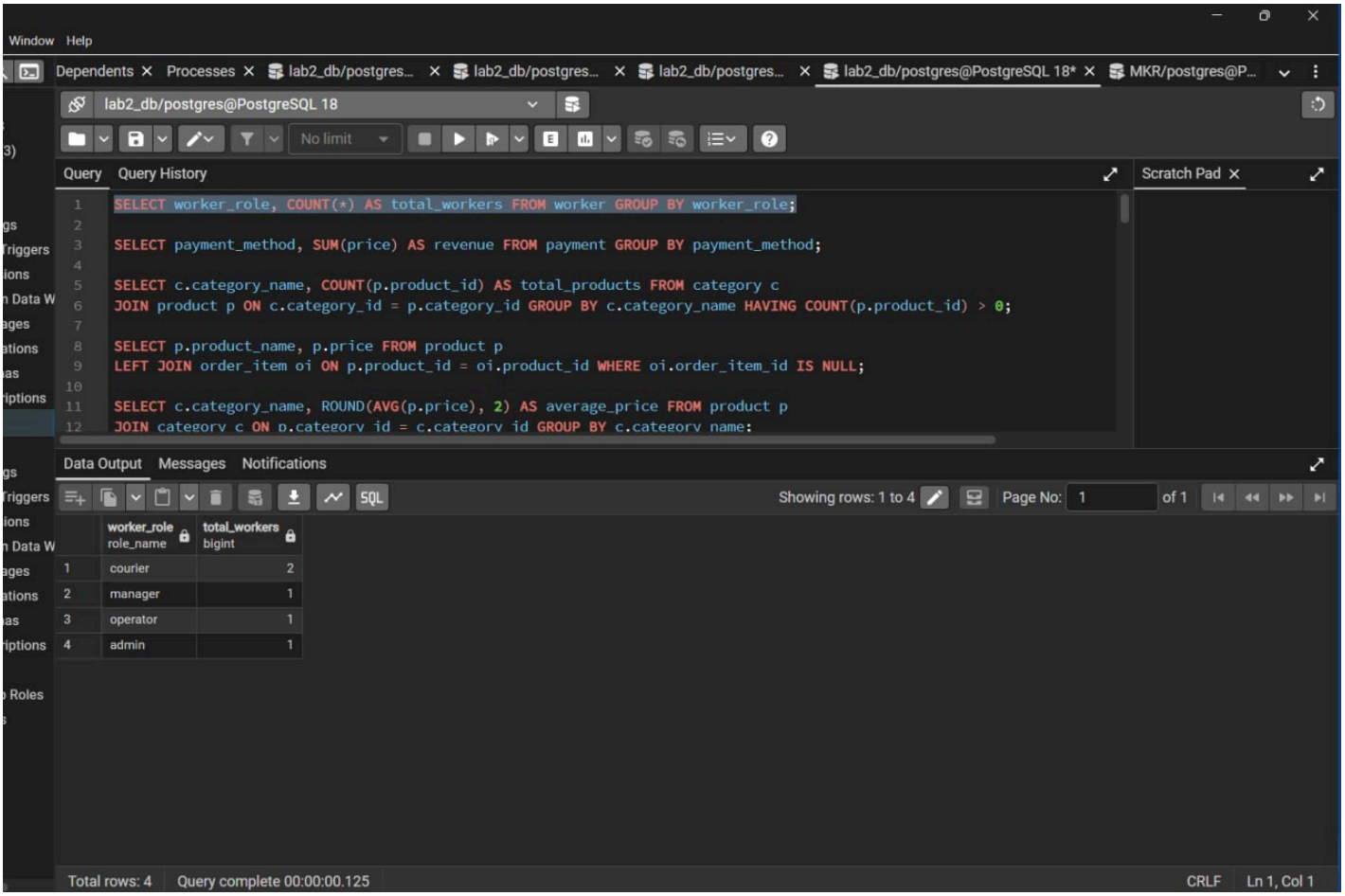
        LEFT JOIN orders o ON a.address_id = o.address_id

        GROUP BY a.city

) AS subquery

WHERE avg_per_person > 1000;
```

Результати виконання:



```
SELECT worker_role, COUNT(*) AS total_workers FROM worker GROUP BY worker_role;

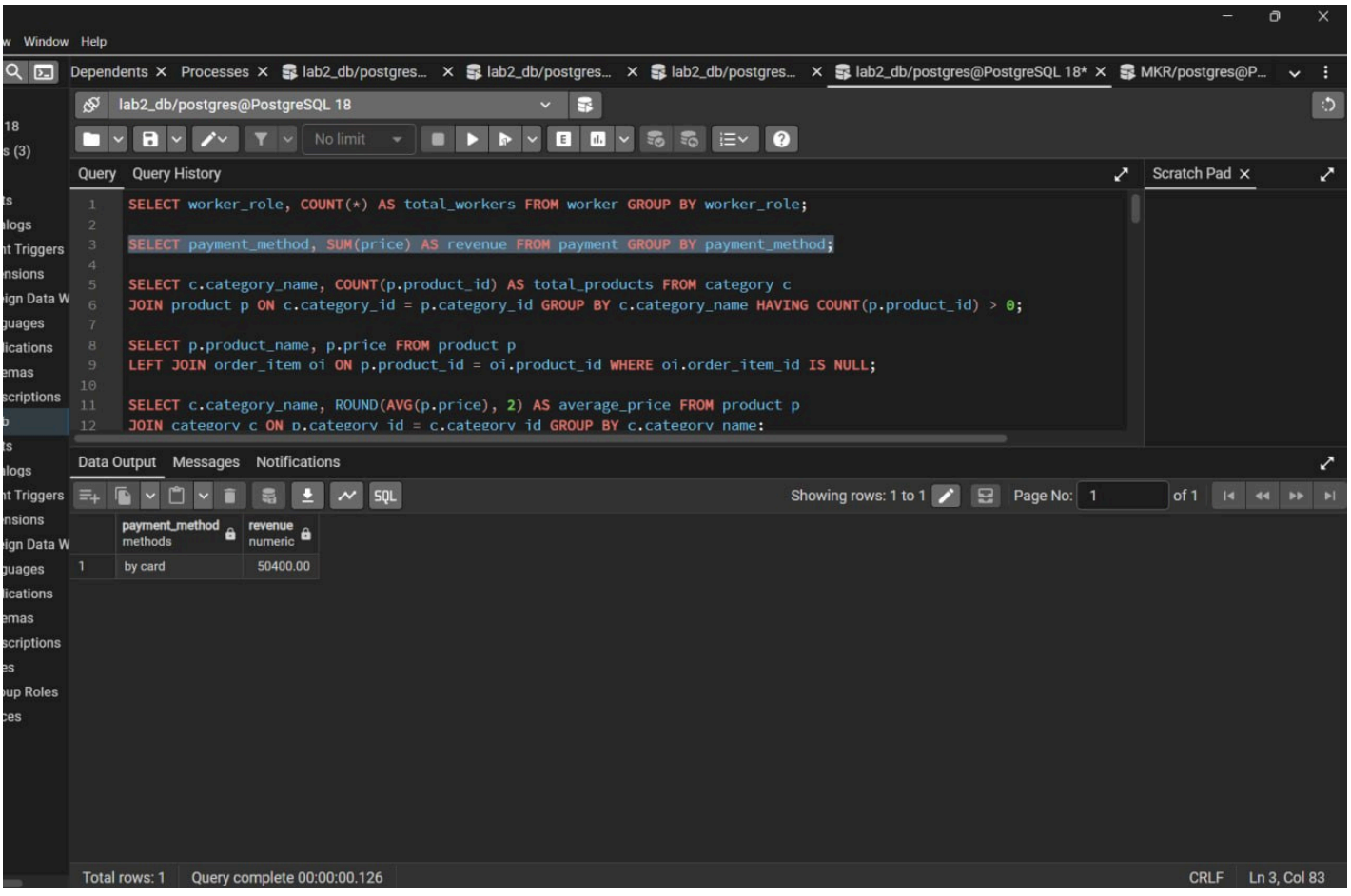
SELECT payment_method, SUM(price) AS revenue FROM payment GROUP BY payment_method;

SELECT c.category_name, COUNT(p.product_id) AS total_products FROM category c
JOIN product p ON c.category_id = p.category_id GROUP BY c.category_name HAVING COUNT(p.product_id) > 0;

SELECT p.product_name, p.price FROM product p
LEFT JOIN order_item oi ON p.product_id = oi.product_id WHERE oi.order_item_id IS NULL;

SELECT c.category_name, ROUND(AVG(p.price), 2) AS average_price FROM product p
JOIN category c ON p.category_id = c.category_id GROUP BY c.category_name;
```

	worker_role role_name	total_workers bigint
1	courier	2
2	manager	1
3	operator	1
4	admin	1



```
SELECT worker_role, COUNT(*) AS total_workers FROM worker GROUP BY worker_role;

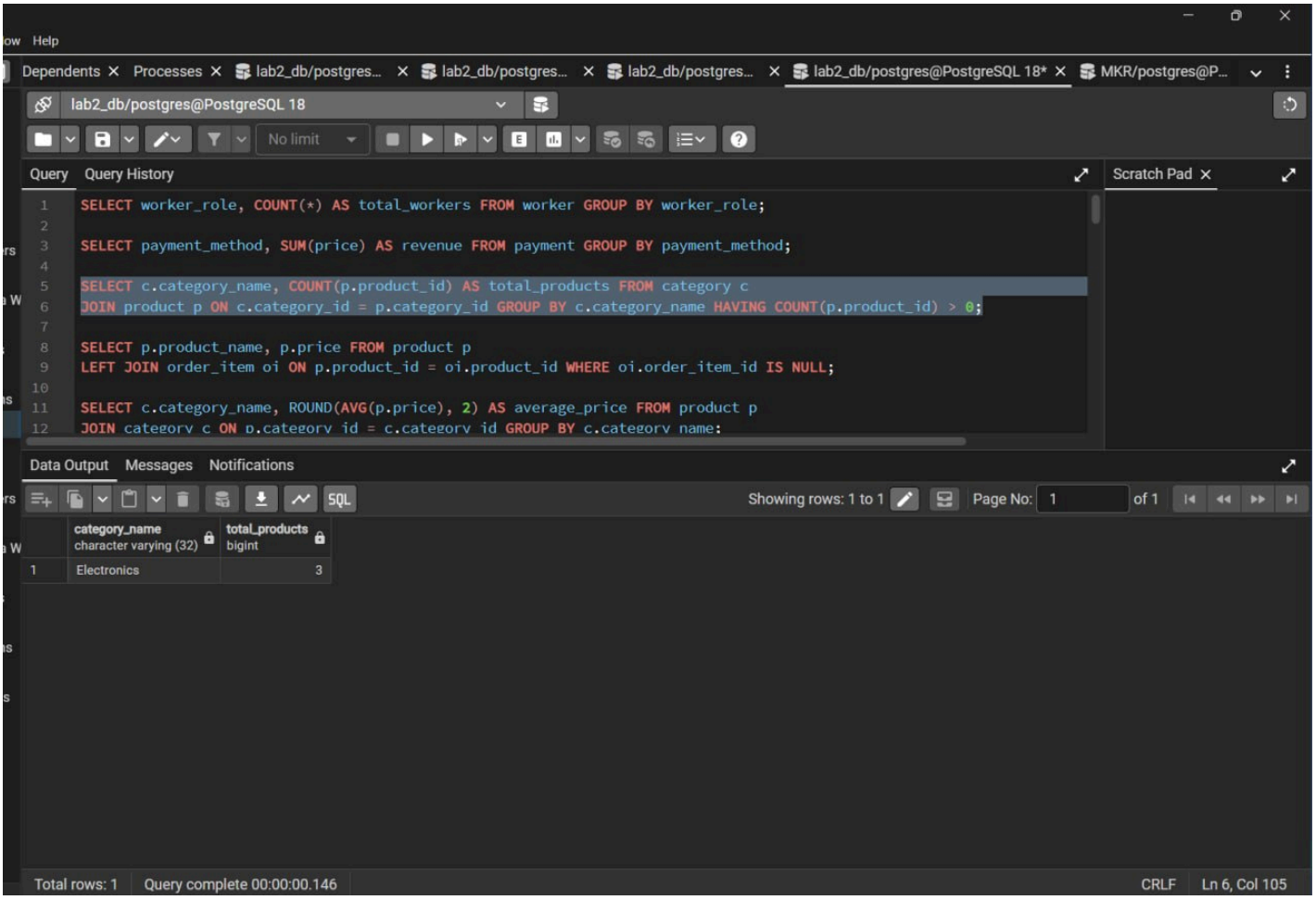
SELECT payment_method, SUM(price) AS revenue FROM payment GROUP BY payment_method;

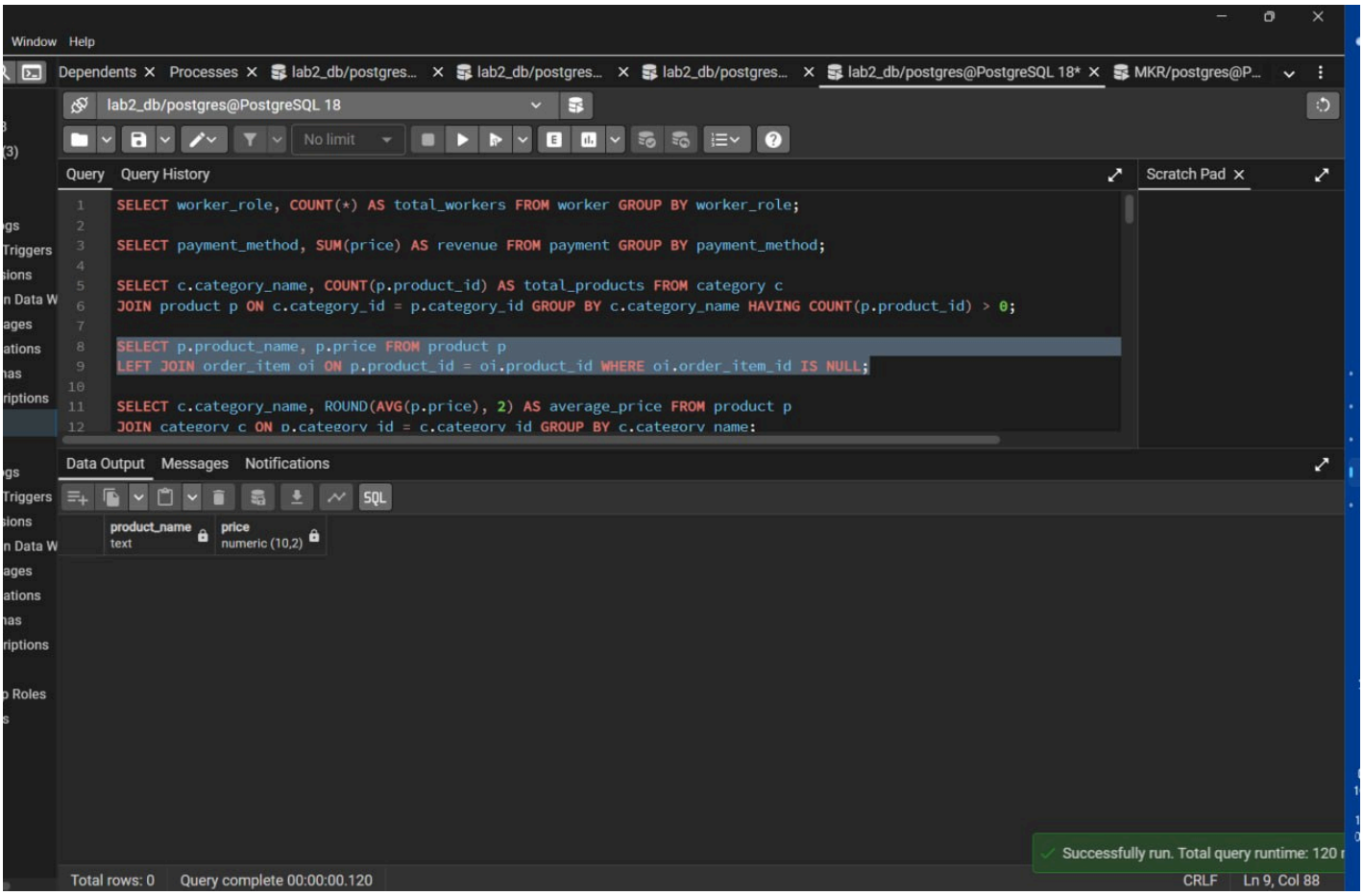
SELECT c.category_name, COUNT(p.product_id) AS total_products FROM category c
JOIN product p ON c.category_id = p.category_id GROUP BY c.category_name HAVING COUNT(p.product_id) > 0;

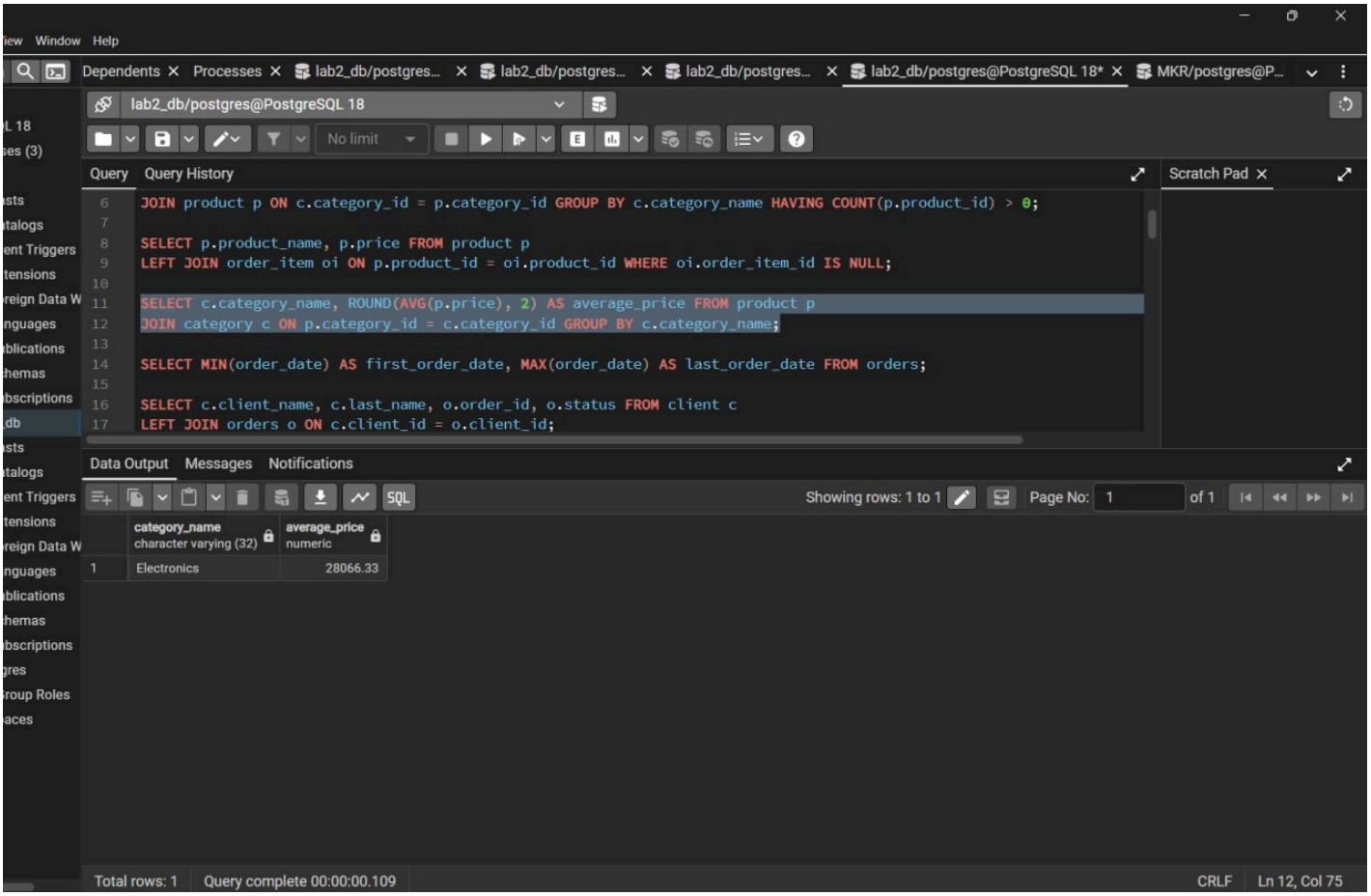
SELECT p.product_name, p.price FROM product p
LEFT JOIN order_item oi ON p.product_id = oi.product_id WHERE oi.order_item_id IS NULL;

SELECT c.category_name, ROUND(AVG(p.price), 2) AS average_price FROM product p
JOIN category c ON p.category_id = c.category_id GROUP BY c.category_name;
```

	payment_method methods	revenue numeric
1	by card	50400.00


Help
Dependents
Processes
lab2_db/postgres...
lab2_db/postgres...
lab2_db/postgres...
lab2_db/postgres@PostgreSQL 18*
MKR/postgres@P...
lab2_db/postgres@PostgreSQL 18
No limit
Query
Query History
Scratch Pad
SELECT worker_role, COUNT(*) AS total_workers FROM worker GROUP BY worker_role;
SELECT payment_method, SUM(price) AS revenue FROM payment GROUP BY payment_method;
SELECT c.category_name, COUNT(p.product_id) AS total_products FROM category c
JOIN product p ON c.category_id = p.category_id GROUP BY c.category_name HAVING COUNT(p.product_id) > 0;
SELECT p.product_name, p.price FROM product p
LEFT JOIN order_item oi ON p.product_id = oi.product_id WHERE oi.order_item_id IS NULL;
SELECT c.category_name, ROUND(AVG(p.price), 2) AS average_price FROM product p
JOIN category c ON p.category_id = c.category_id GROUP BY c.category_name;
Data Output
Messages
Notifications
SQL
Showing rows: 1 to 1
Page No: 1
of 1
category_name
character varying (32)
total_products
bigint
1
Electronics
3
Total rows: 1
Query complete 00:00:00.146
CRLF
Ln 6, Col 105


Window
Help
Dependents
Processes
lab2_db/postgres...
lab2_db/postgres...
lab2_db/postgres...
lab2_db/postgres@PostgreSQL 18*
MKR/postgres@P...
lab2_db/postgres@PostgreSQL 18
No limit
Query
Query History
Scratch Pad
SELECT worker_role, COUNT(*) AS total_workers FROM worker GROUP BY worker_role;
SELECT payment_method, SUM(price) AS revenue FROM payment GROUP BY payment_method;
SELECT c.category_name, COUNT(p.product_id) AS total_products FROM category c
JOIN product p ON c.category_id = p.category_id GROUP BY c.category_name HAVING COUNT(p.product_id) > 0;
SELECT p.product_name, p.price FROM product p
LEFT JOIN order_item oi ON p.product_id = oi.product_id WHERE oi.order_item_id IS NULL;
SELECT c.category_name, ROUND(AVG(p.price), 2) AS average_price FROM product p
JOIN category c ON p.category_id = c.category_id GROUP BY c.category_name;
Data Output
Messages
Notifications
SQL
product_name
text
price
numeric (10,2)
Successfully run. Total query runtime: 120
Total rows: 0
Query complete 00:00:00.120
CRLF
Ln 9, Col 88



```
JOIN product p ON c.category_id = p.category_id GROUP BY c.category_name HAVING COUNT(p.product_id) > 0;

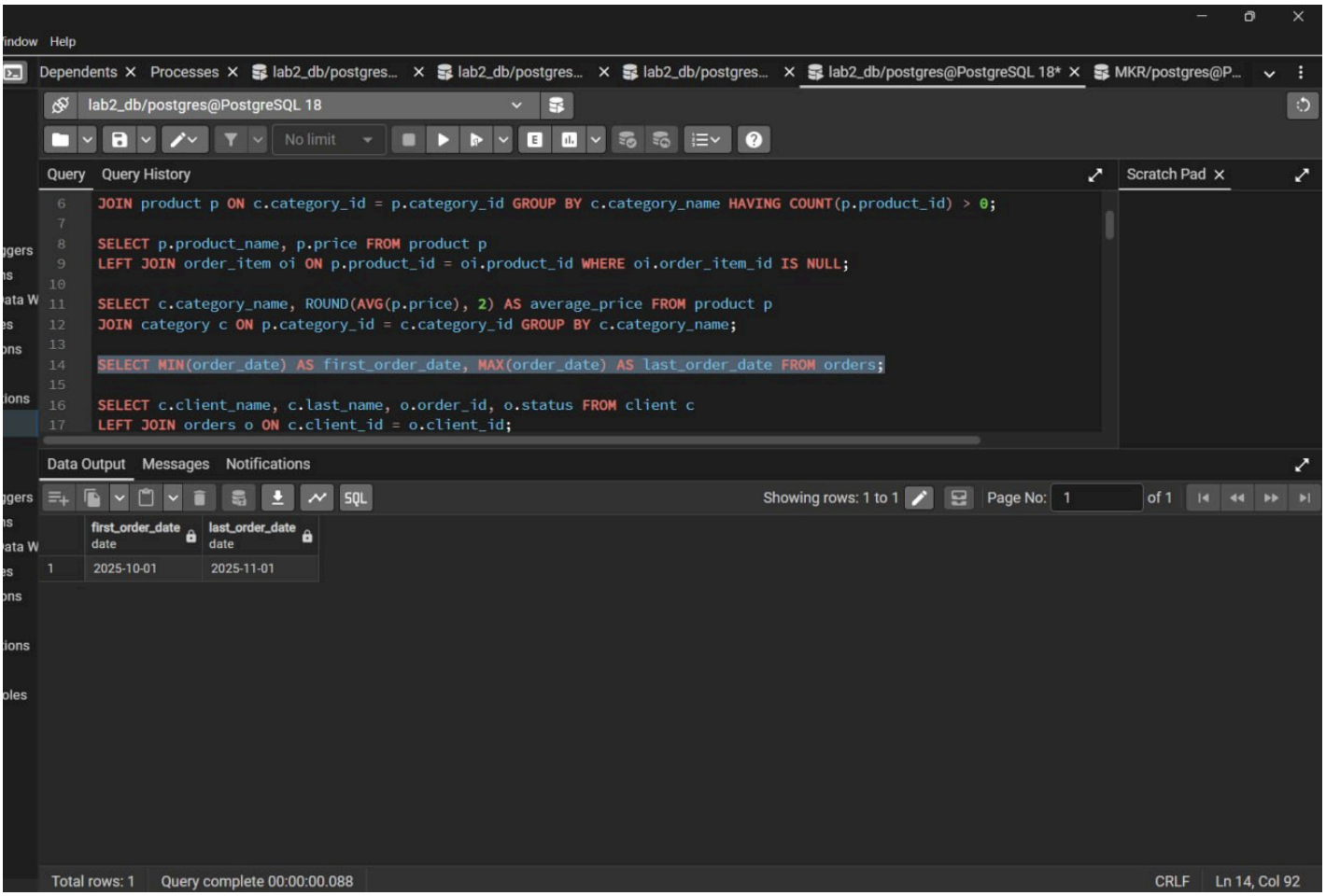
SELECT p.product_name, p.price FROM product p
LEFT JOIN order_item oi ON p.product_id = oi.product_id WHERE oi.order_item_id IS NULL;

SELECT c.category_name, ROUND(AVG(p.price), 2) AS average_price FROM product p
JOIN category c ON p.category_id = c.category_id GROUP BY c.category_name;

SELECT MIN(order_date) AS first_order_date, MAX(order_date) AS last_order_date FROM orders;

SELECT c.client_name, c.last_name, o.order_id, o.status FROM client c
LEFT JOIN orders o ON c.client_id = o.client_id;
```

	category_name character varying (32)	average_price numeric
1	Electronics	28066.33



```
JOIN product p ON c.category_id = p.category_id GROUP BY c.category_name HAVING COUNT(p.product_id) > 0;

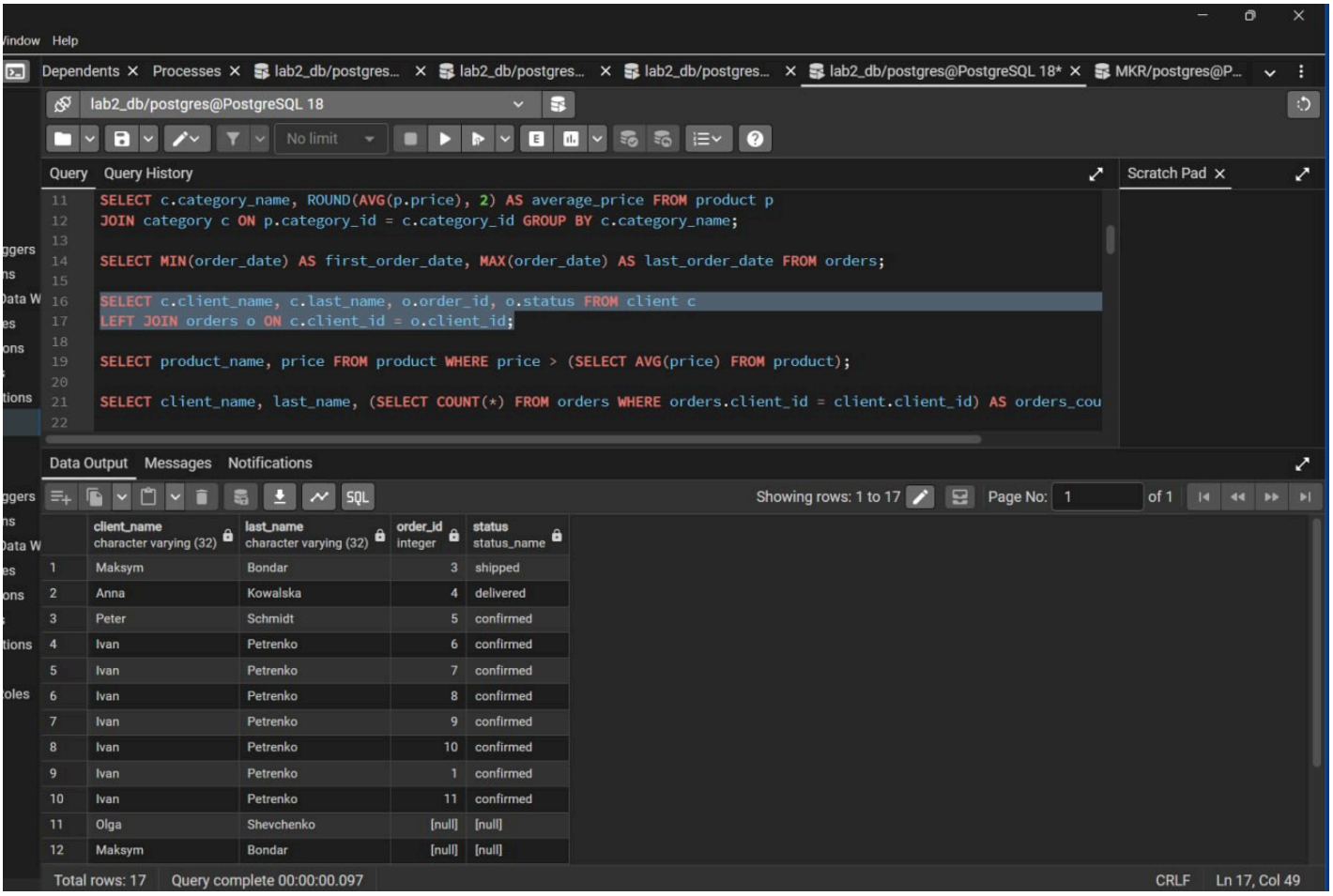
SELECT p.product_name, p.price FROM product p
LEFT JOIN order_item oi ON p.product_id = oi.product_id WHERE oi.order_item_id IS NULL;

SELECT c.category_name, ROUND(AVG(p.price), 2) AS average_price FROM product p
JOIN category c ON p.category_id = c.category_id GROUP BY c.category_name;

SELECT MIN(order_date) AS first_order_date, MAX(order_date) AS last_order_date FROM orders;

SELECT c.client_name, c.last_name, o.order_id, o.status FROM client c
LEFT JOIN orders o ON c.client_id = o.client_id;
```

	first_order_date date	last_order_date date
1	2025-10-01	2025-11-01



```
SELECT c.category_name, ROUND(AVG(p.price), 2) AS average_price FROM product p
JOIN category c ON p.category_id = c.category_id GROUP BY c.category_name;

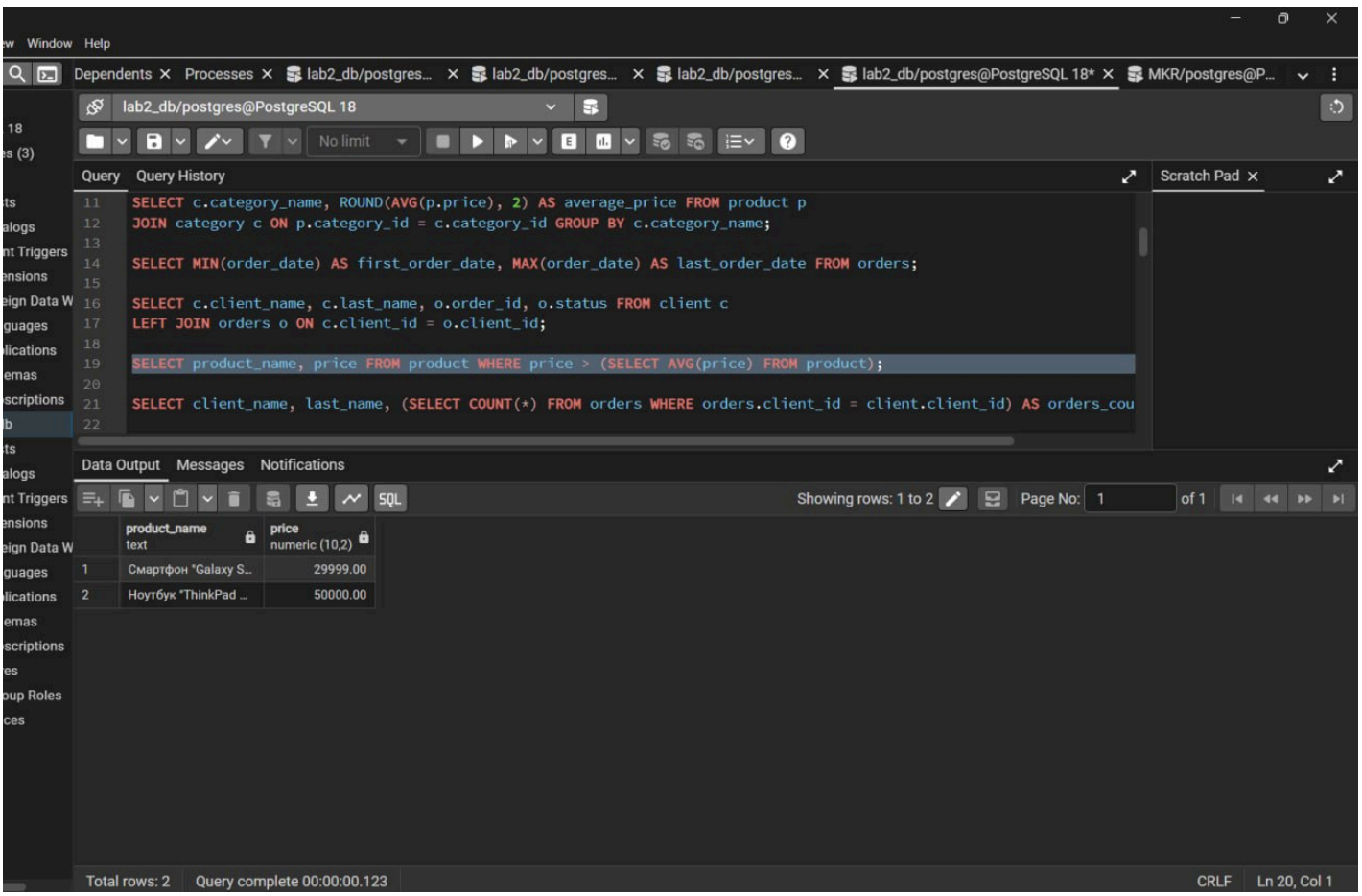
SELECT MIN(order_date) AS first_order_date, MAX(order_date) AS last_order_date FROM orders;

SELECT c.client_name, c.last_name, o.order_id, o.status FROM client c
LEFT JOIN orders o ON c.client_id = o.client_id;

SELECT product_name, price FROM product WHERE price > (SELECT AVG(price) FROM product);

SELECT client_name, last_name, (SELECT COUNT(*) FROM orders WHERE orders.client_id = client.client_id) AS orders_cou
```

	client_name character varying (32)	last_name character varying (32)	order_id integer	status status_name
1	Maksym	Bondar	3	shipped
2	Anna	Kowalska	4	delivered
3	Peter	Schmidt	5	confirmed
4	Ivan	Petrenko	6	confirmed
5	Ivan	Petrenko	7	confirmed
6	Ivan	Petrenko	8	confirmed
7	Ivan	Petrenko	9	confirmed
8	Ivan	Petrenko	10	confirmed
9	Ivan	Petrenko	1	confirmed
10	Ivan	Petrenko	11	confirmed
11	Olga	Shevchenko	[null]	[null]
12	Maksym	Bondar	[null]	[null]



```
SELECT c.category_name, ROUND(AVG(p.price), 2) AS average_price FROM product p
JOIN category c ON p.category_id = c.category_id GROUP BY c.category_name;

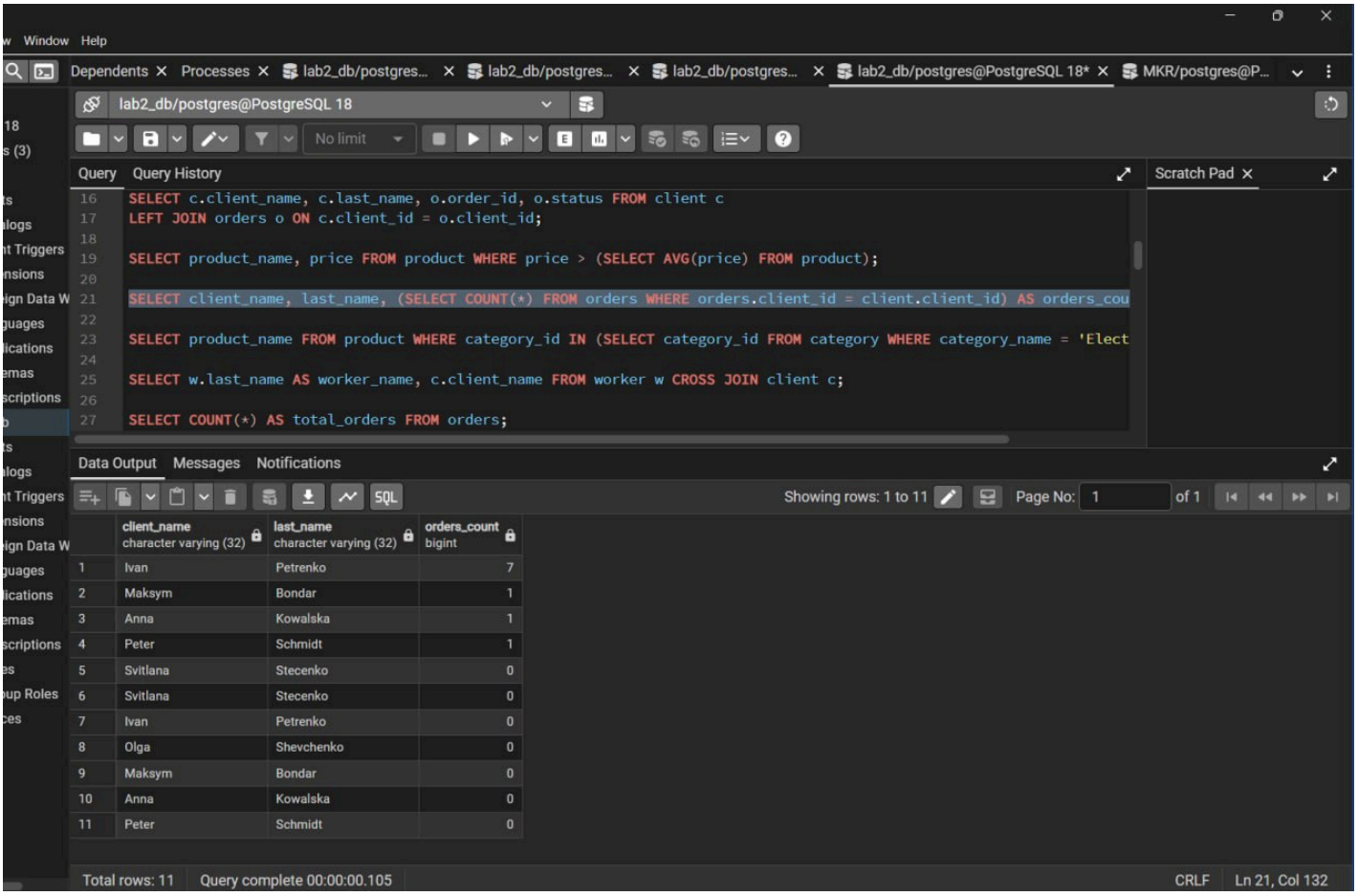
SELECT MIN(order_date) AS first_order_date, MAX(order_date) AS last_order_date FROM orders;

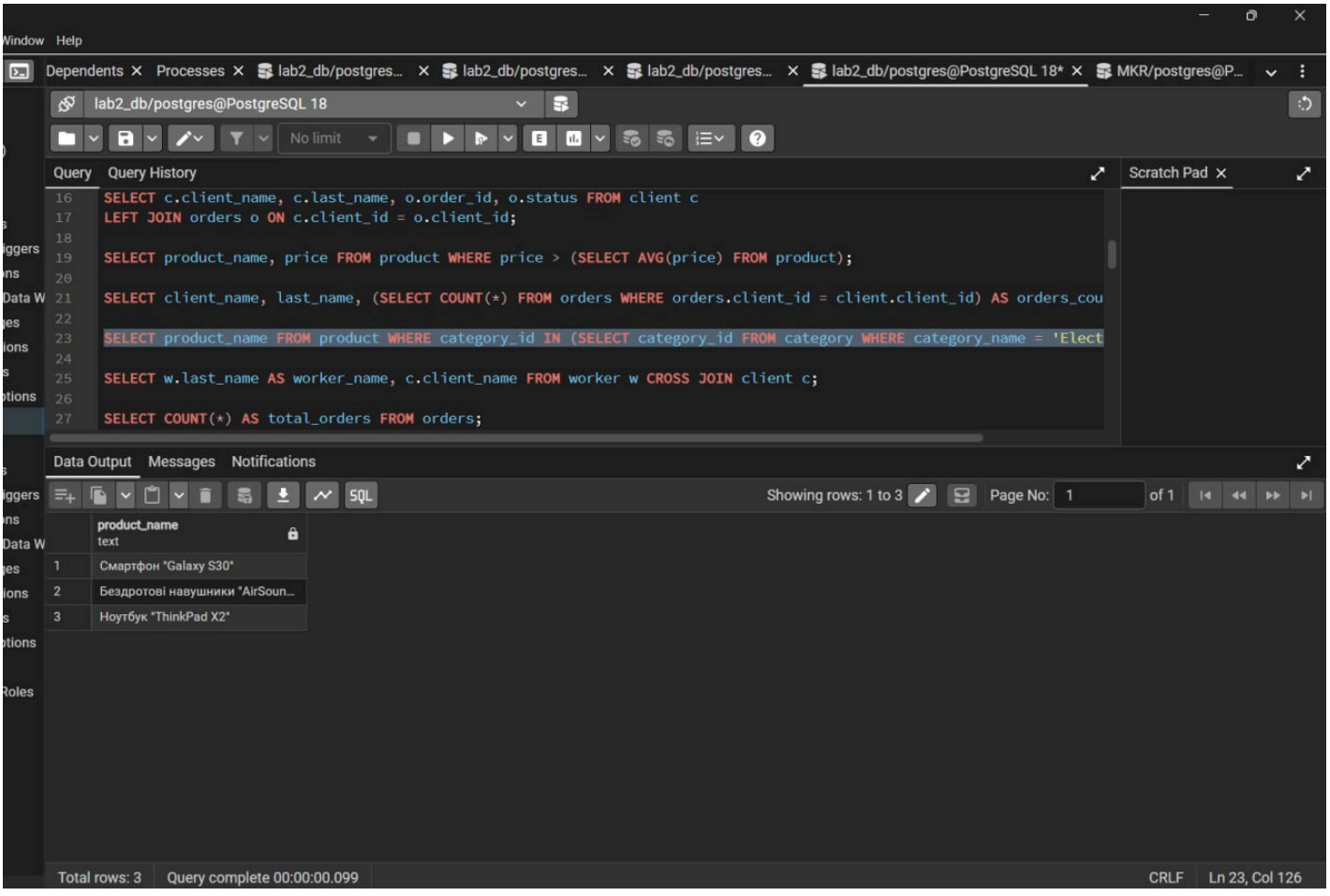
SELECT c.client_name, c.last_name, o.order_id, o.status FROM client c
LEFT JOIN orders o ON c.client_id = o.client_id;

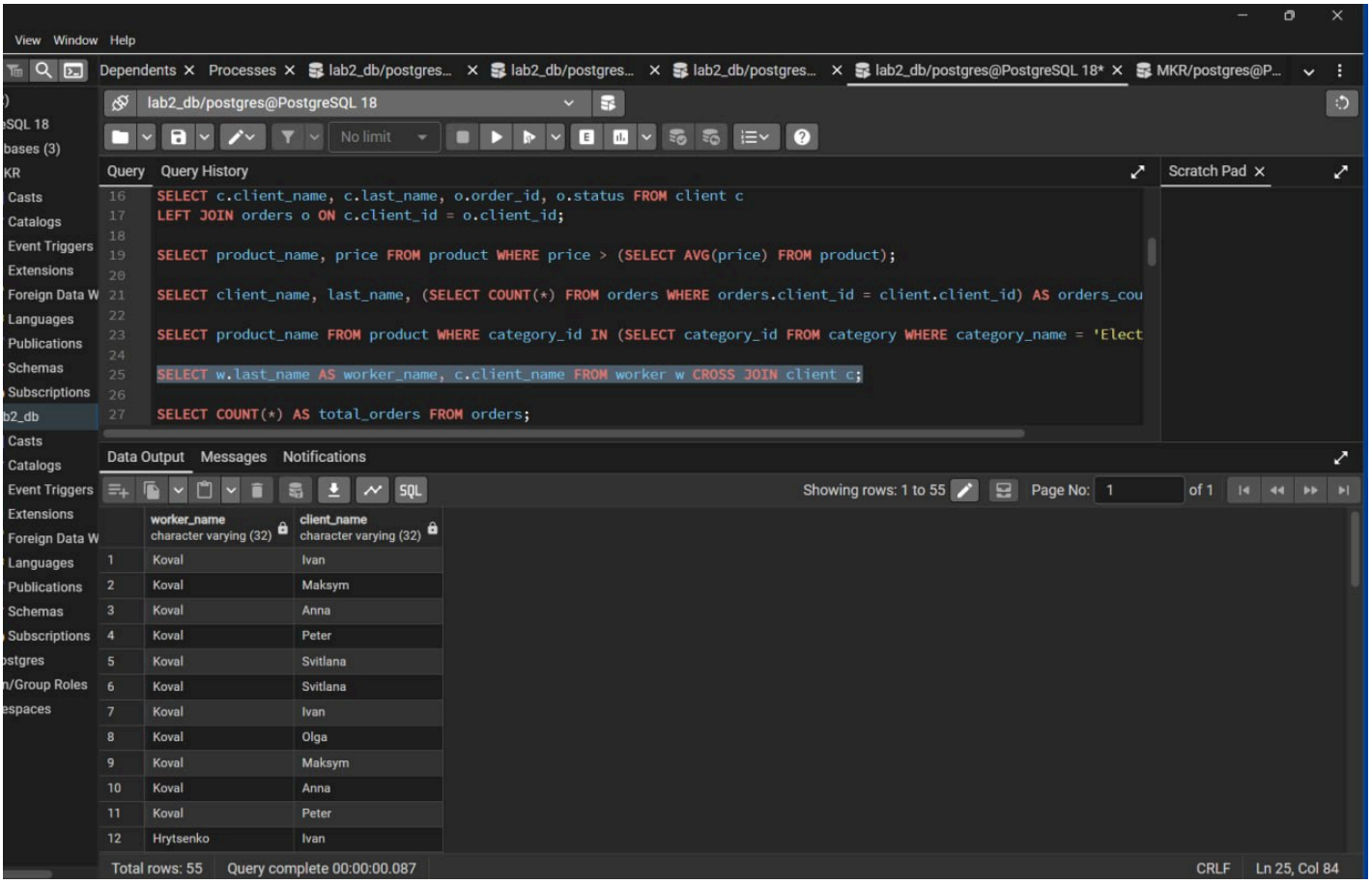
SELECT product_name, price FROM product WHERE price > (SELECT AVG(price) FROM product);

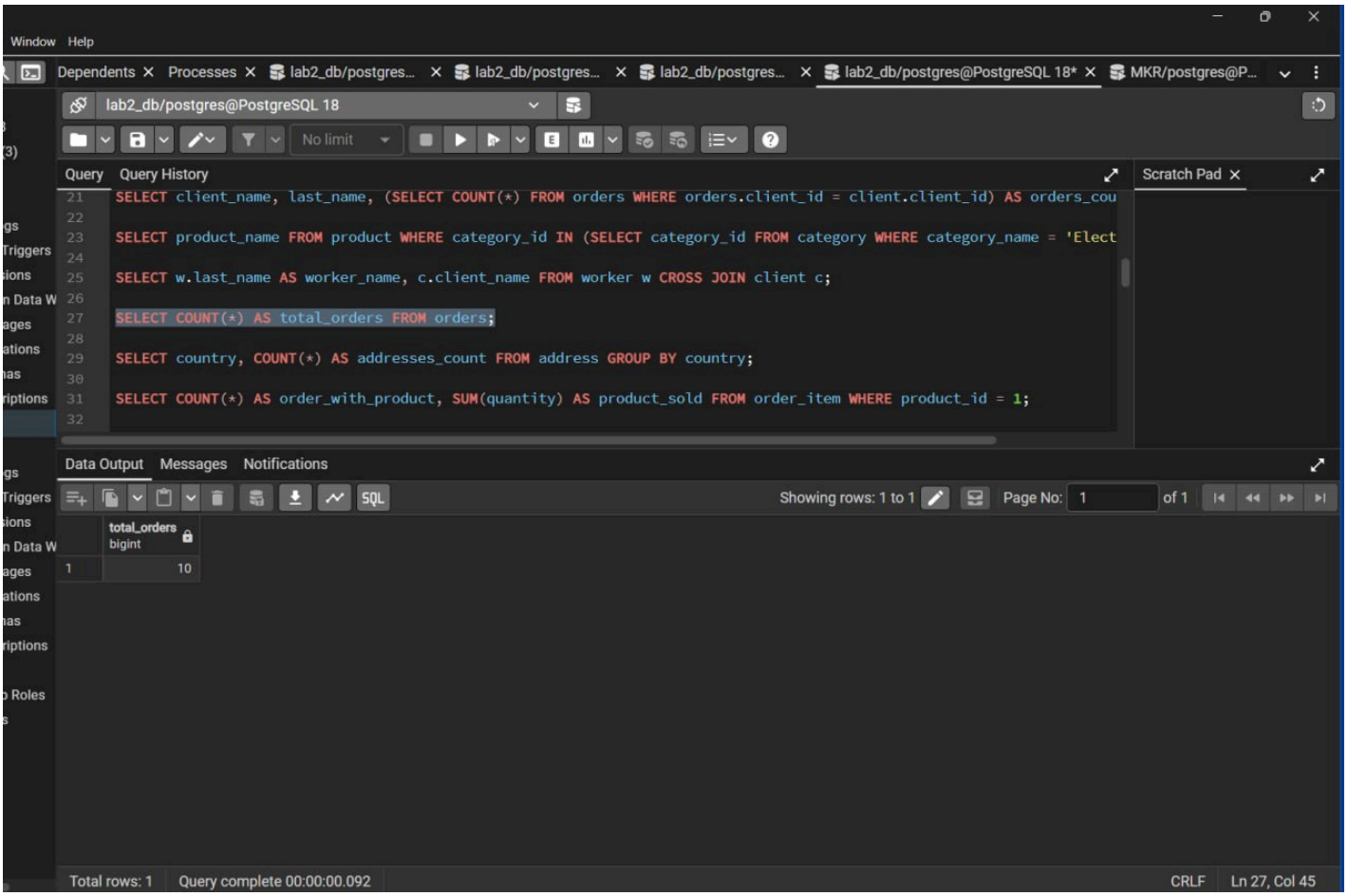
SELECT client_name, last_name, (SELECT COUNT(*) FROM orders WHERE orders.client_id = client.client_id) AS orders_cou
```

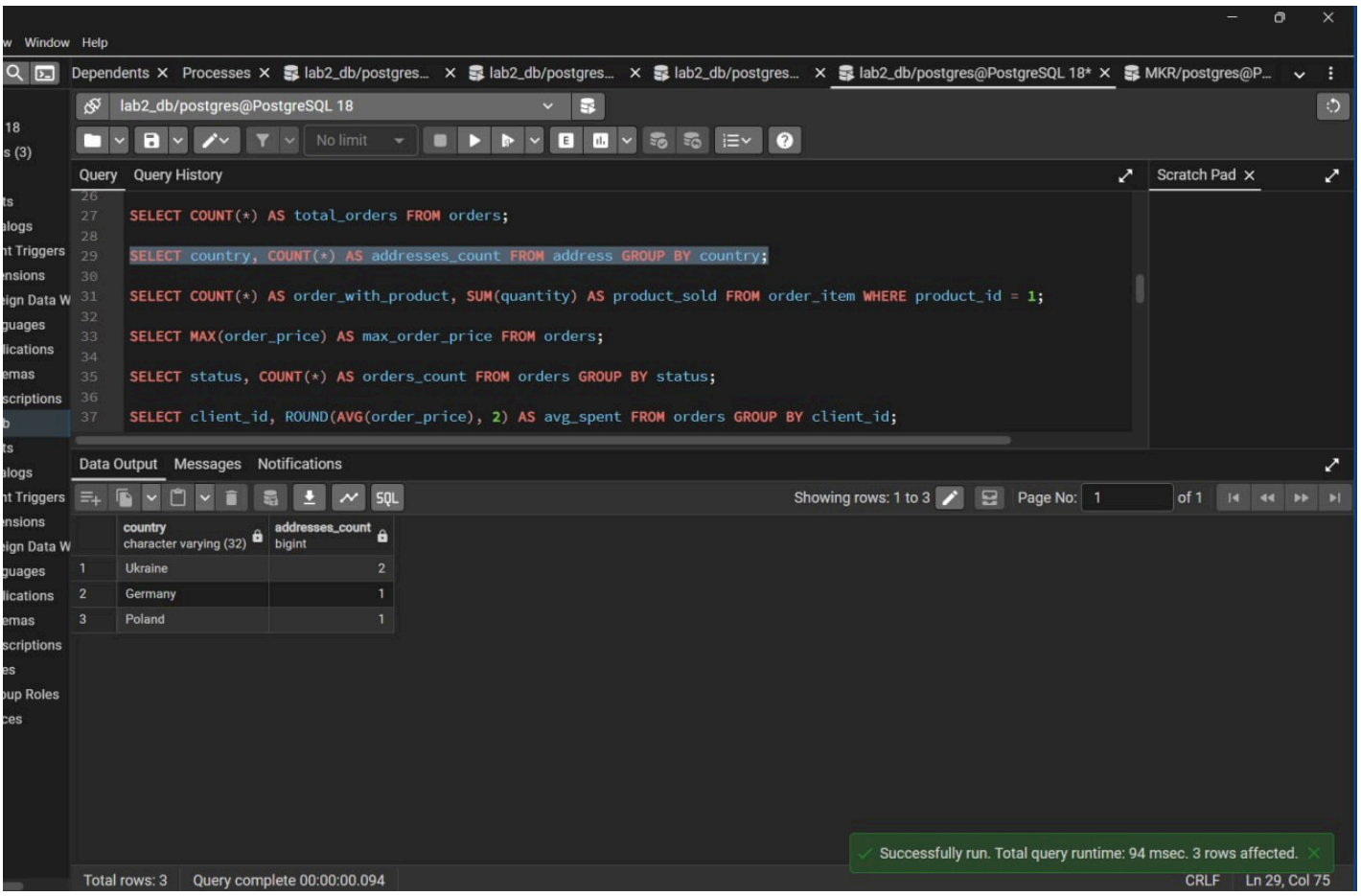
	product_name text	price numeric (10,2)
1	Смартфон "Galaxy S...	29999.00
2	Ноутбук "ThinkPad ...	50000.00

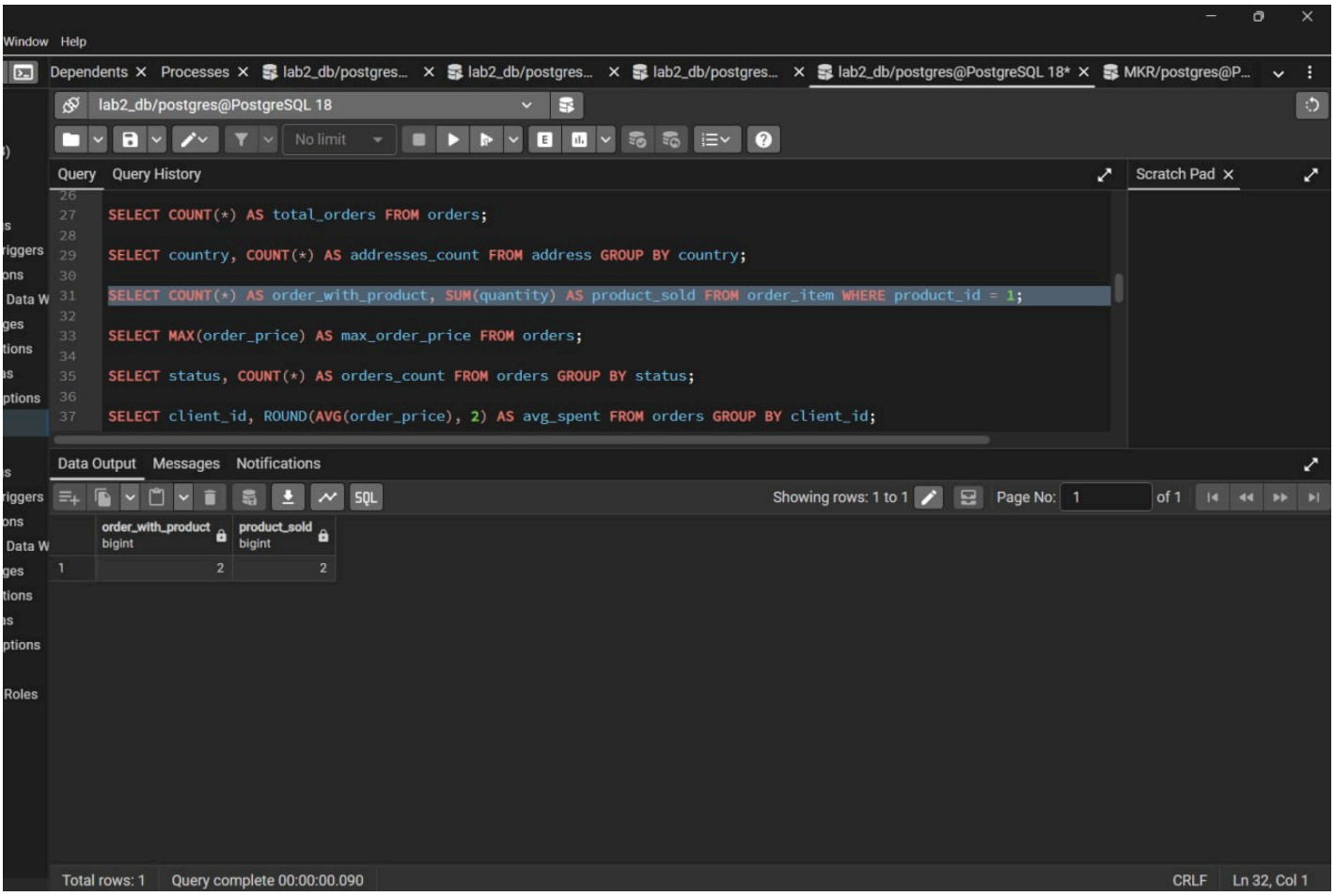

Window Help
Dependents
Processes
lab2_db/postgres...
lab2_db/postgres...
lab2_db/postgres...
lab2_db/postgres@PostgreSQL 18*
MKR/postgres@P...
lab2_db/postgres@PostgreSQL 18
No limit
Query
Query History
Scratch Pad
16 SELECT c.client_name, c.last_name, o.order_id, o.status FROM client c
17 LEFT JOIN orders o ON c.client_id = o.client_id;
18
19 SELECT product_name, price FROM product WHERE price > (SELECT AVG(price) FROM product);
20
21 SELECT client_name, last_name, (SELECT COUNT(*) FROM orders WHERE orders.client_id = client.client_id) AS orders_cou
22
23 SELECT product_name FROM product WHERE category_id IN (SELECT category_id FROM category WHERE category_name = 'Elect
24
25 SELECT w.last_name AS worker_name, c.client_name FROM worker w CROSS JOIN client c;
26
27 SELECT COUNT(*) AS total_orders FROM orders;
Data Output
Messages
Notifications
SQL
Showing rows: 1 to 11
Page No: 1 of 1
client_name character varying (32)
last_name character varying (32)
orders_count bigint
1 Ivan Petrenko 7
2 Maksym Bondar 1
3 Anna Kowalska 1
4 Peter Schmidt 1
5 Svitlana Stecenko 0
6 Svitlana Stecenko 0
7 Ivan Petrenko 0
8 Olga Shevchenko 0
9 Maksym Bondar 0
10 Anna Kowalska 0
11 Peter Schmidt 0
Total rows: 11
Query complete 00:00:00.105
CRLF
Ln 21, Col 132

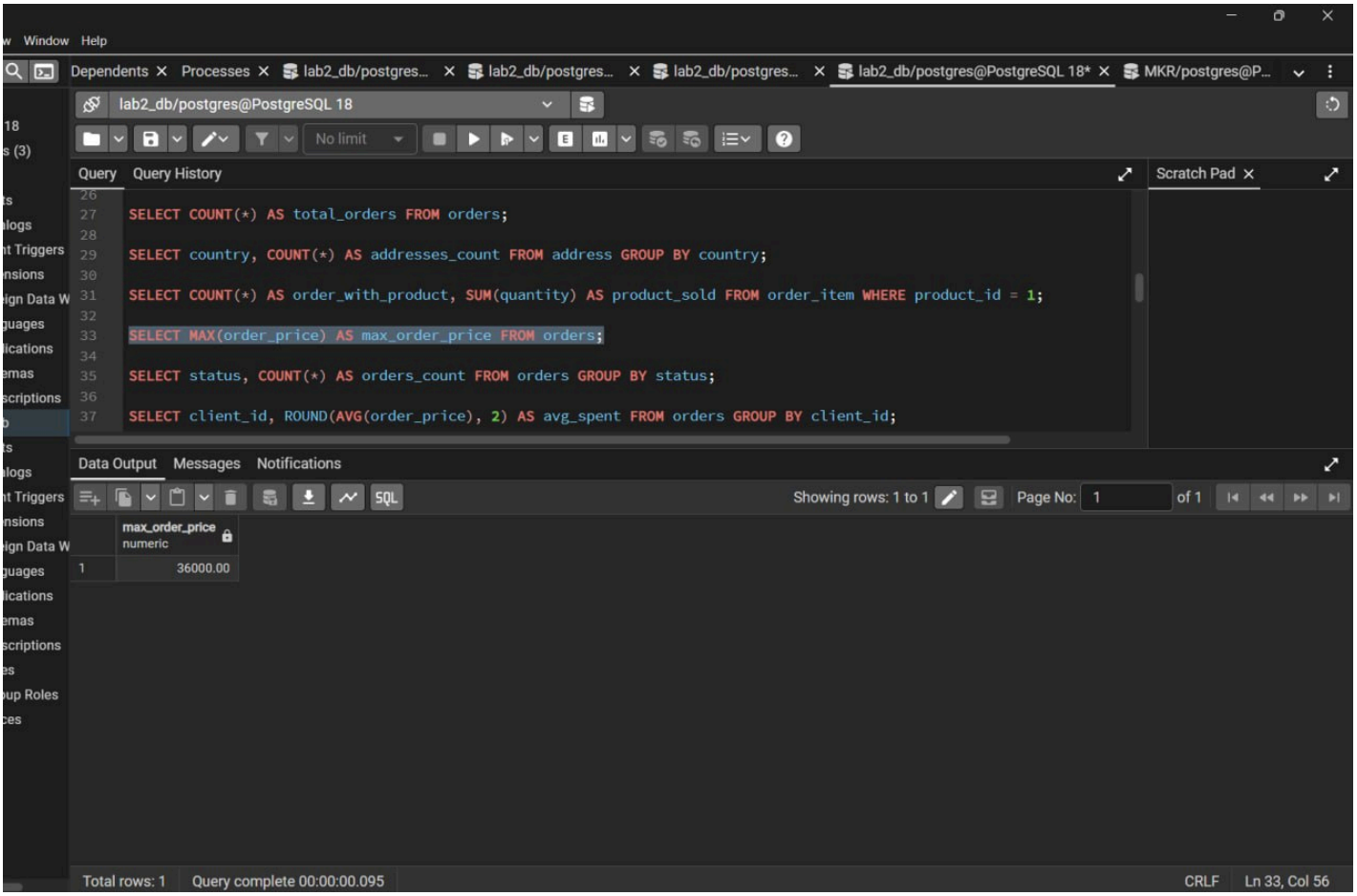

Window Help
Dependents
Processes
lab2_db/postgres...
lab2_db/postgres...
lab2_db/postgres...
lab2_db/postgres@PostgreSQL 18*
MKR/postgres@P...
lab2_db/postgres@PostgreSQL 18
No limit
Query
Query History
Scratch Pad
16 SELECT c.client_name, c.last_name, o.order_id, o.status FROM client c
17 LEFT JOIN orders o ON c.client_id = o.client_id;
18
19 SELECT product_name, price FROM product WHERE price > (SELECT AVG(price) FROM product);
20
21 SELECT client_name, last_name, (SELECT COUNT(*) FROM orders WHERE orders.client_id = client.client_id) AS orders_cou
22
23 SELECT product_name FROM product WHERE category_id IN (SELECT category_id FROM category WHERE category_name = 'Elect
24
25 SELECT w.last_name AS worker_name, c.client_name FROM worker w CROSS JOIN client c;
26
27 SELECT COUNT(*) AS total_orders FROM orders;
Data Output
Messages
Notifications
SQL
Showing rows: 1 to 3
Page No: 1 of 1
product_name text
1 Смартфон "Galaxy S30"
2 Бездротові навушники "AirSoun...
3 Ноутбук "ThinkPad X2"
Total rows: 3
Query complete 00:00:00.099
CRLF
Ln 23, Col 126

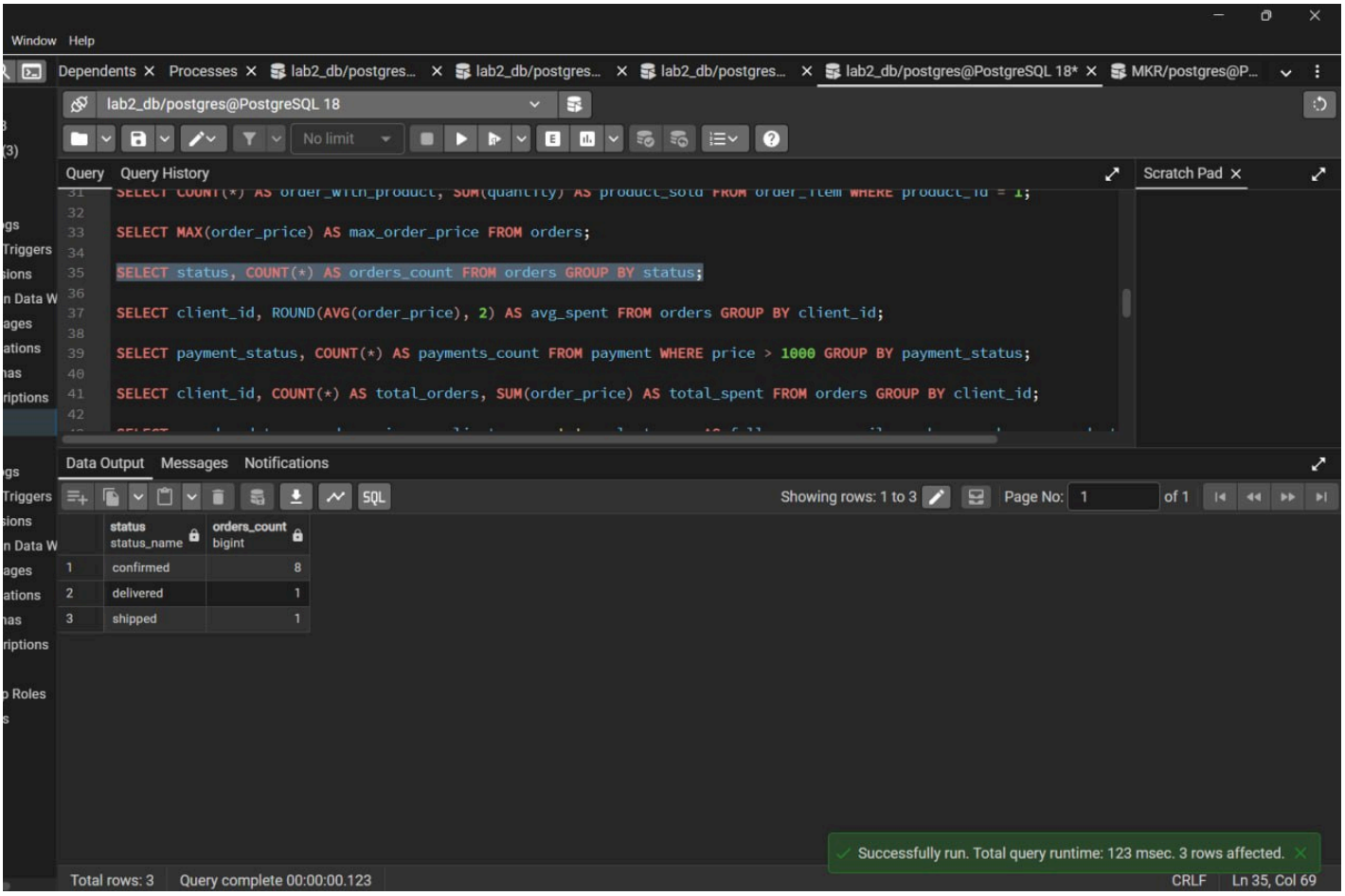

View Window Help
Dependents Processes lab2_db/postgres... lab2_db/postgres... lab2_db/postgres... lab2_db/postgres@PostgreSQL 18* MKR/postgres@P...
lab2_db/postgres@PostgreSQL 18
No limit
Query Query History
Scratch Pad
16 SELECT c.client_name, c.last_name, o.order_id, o.status FROM client c
17 LEFT JOIN orders o ON c.client_id = o.client_id;
18
19 SELECT product_name, price FROM product WHERE price > (SELECT AVG(price) FROM product);
20
21 SELECT client_name, last_name, (SELECT COUNT(*) FROM orders WHERE orders.client_id = client.client_id) AS orders_cou
22
23 SELECT product_name FROM product WHERE category_id IN (SELECT category_id FROM category WHERE category_name = 'Elect
24
25 SELECT w.last_name AS worker_name, c.client_name FROM worker w CROSS JOIN client c;
26
27 SELECT COUNT(*) AS total_orders FROM orders;
Data Output Messages Notifications
Showing rows: 1 to 55 Page No: 1 of 1
worker_name character varying (32) client_name character varying (32)
1 Koval Ivan
2 Koval Maksym
3 Koval Anna
4 Koval Peter
5 Koval Svitlana
6 Koval Svitlana
7 Koval Ivan
8 Koval Olga
9 Koval Maksym
10 Koval Anna
11 Koval Peter
12 Hrytsenko Ivan
Total rows: 55 Query complete 00:00:00.087
CRLF Ln 25, Col 84

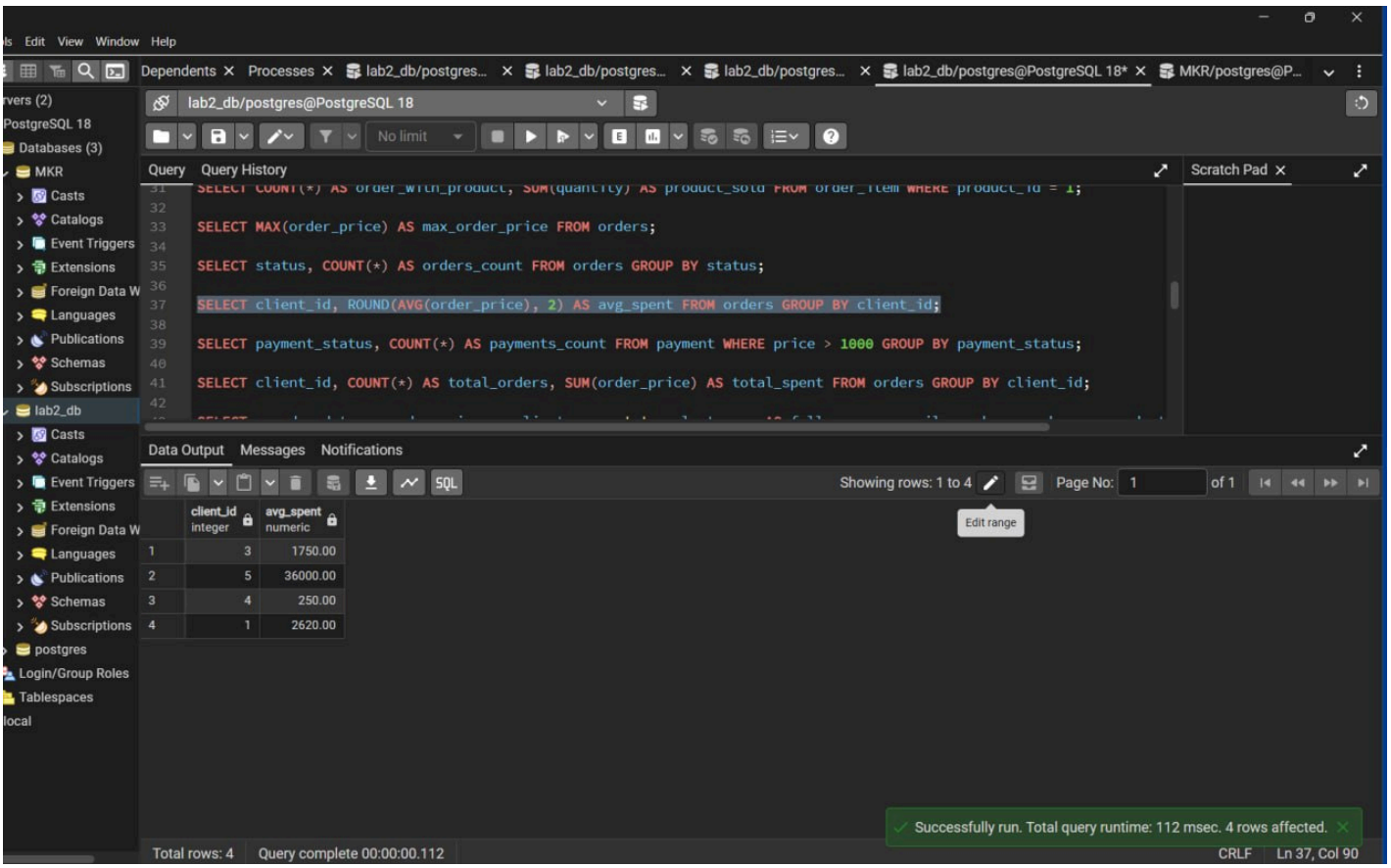

Window Help
Dependents Processes lab2_db/postgres... lab2_db/postgres... lab2_db/postgres... lab2_db/postgres@PostgreSQL 18* MKR/postgres@P...
lab2_db/postgres@PostgreSQL 18
No limit
Query Query History
Scratch Pad
21 SELECT client_name, last_name, (SELECT COUNT(*) FROM orders WHERE orders.client_id = client.client_id) AS orders_cou
22
23 SELECT product_name FROM product WHERE category_id IN (SELECT category_id FROM category WHERE category_name = 'Elect
24
25 SELECT w.last_name AS worker_name, c.client_name FROM worker w CROSS JOIN client c;
26
27 SELECT COUNT(*) AS total_orders FROM orders;
28
29 SELECT country, COUNT(*) AS addresses_count FROM address GROUP BY country;
30
31 SELECT COUNT(*) AS order_with_product, SUM(quantity) AS product_sold FROM order_item WHERE product_id = 1;
32
Data Output Messages Notifications
Showing rows: 1 to 1 Page No: 1 of 1
total_orders bigint
1 10
Total rows: 1 Query complete 00:00:00.092
CRLF Ln 27, Col 45

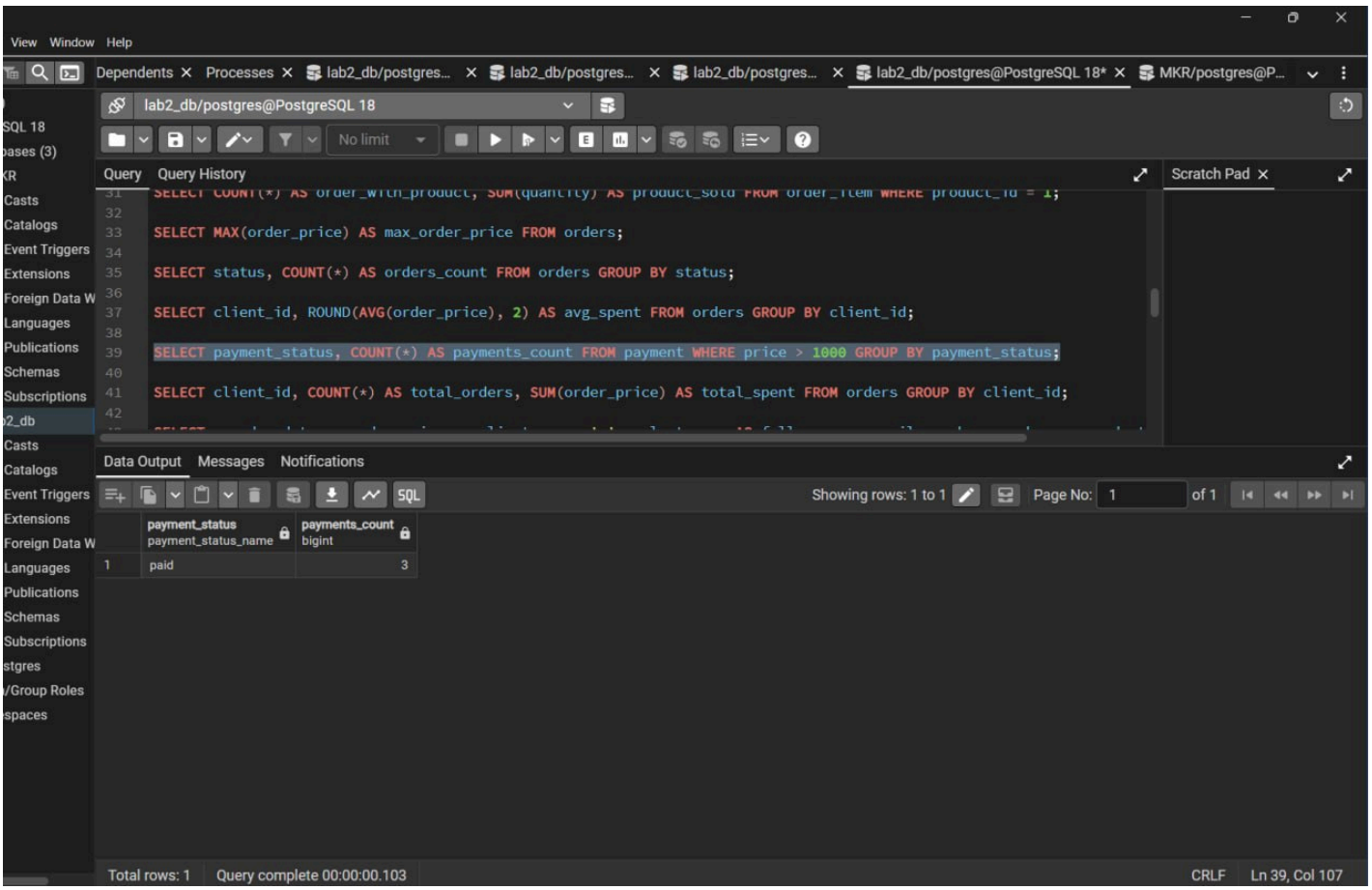

Window Help
Dependents
Processes
lab2_db/postgres...
lab2_db/postgres...
lab2_db/postgres...
lab2_db/postgres@PostgreSQL 18*
MKR/postgres@P...
lab2_db/postgres@PostgreSQL 18
No limit
Query
Query History
Scratch Pad
26
27 SELECT COUNT(*) AS total_orders FROM orders;
28
29 SELECT country, COUNT(*) AS addresses_count FROM address GROUP BY country;
30
31 SELECT COUNT(*) AS order_with_product, SUM(quantity) AS product_sold FROM order_item WHERE product_id = 1;
32
33 SELECT MAX(order_price) AS max_order_price FROM orders;
34
35 SELECT status, COUNT(*) AS orders_count FROM orders GROUP BY status;
36
37 SELECT client_id, ROUND(AVG(order_price), 2) AS avg_spent FROM orders GROUP BY client_id;
Data Output
Messages
Notifications
SQL
Showing rows: 1 to 3
Page No: 1
of 1
country character varying (32)
addresses_count bigint
1 Ukraine 2
2 Germany 1
3 Poland 1
Successfully run. Total query runtime: 94 msec. 3 rows affected.
Total rows: 3
Query complete 00:00:00.094
CRLF
Ln 29, Col 75

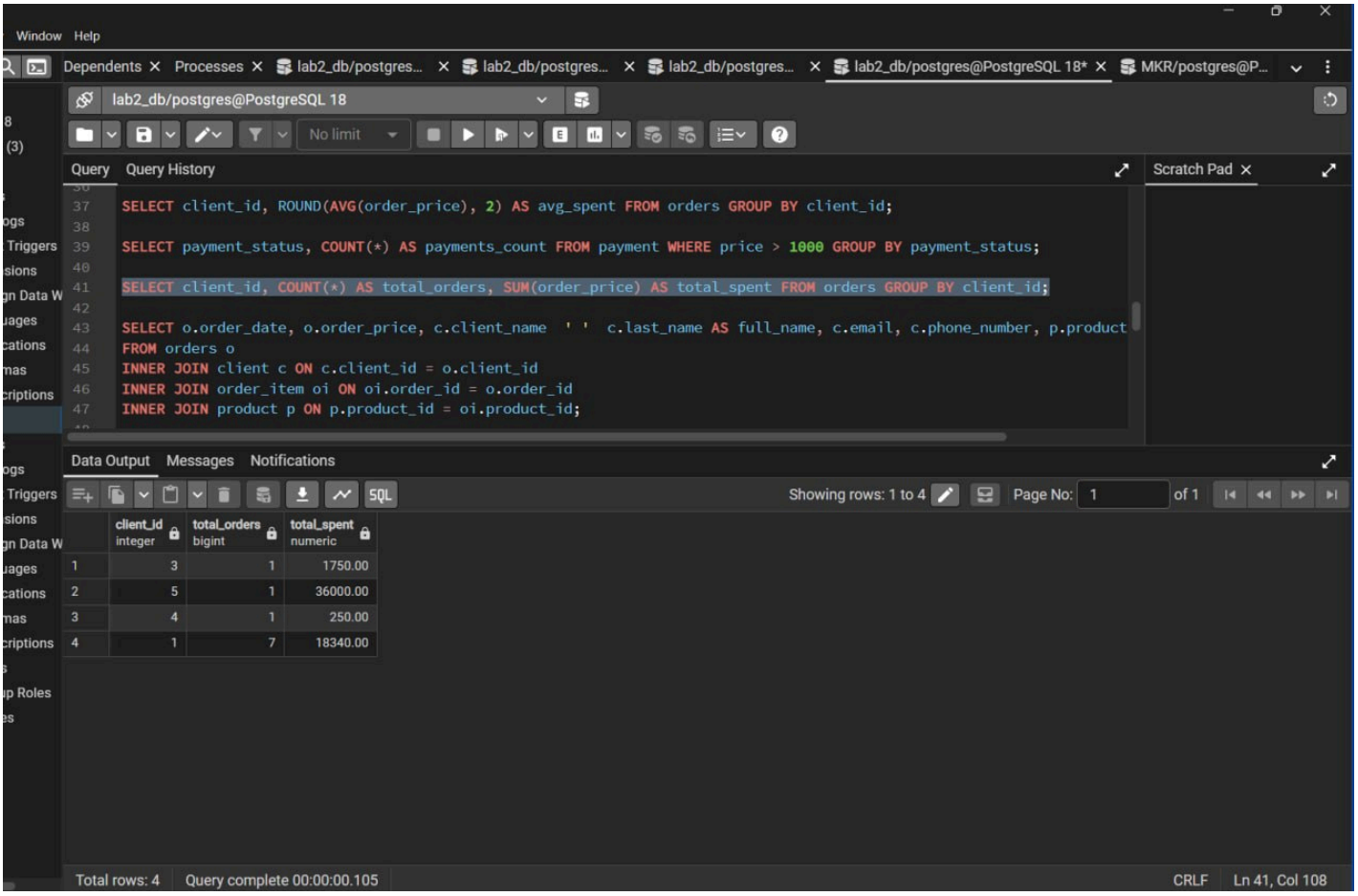

Window Help
Dependents
Processes
lab2_db/postgres...
lab2_db/postgres...
lab2_db/postgres...
lab2_db/postgres@PostgreSQL 18*
MKR/postgres@P...
lab2_db/postgres@PostgreSQL 18
No limit
Query
Query History
Scratch Pad
26
27 SELECT COUNT(*) AS total_orders FROM orders;
28
29 SELECT country, COUNT(*) AS addresses_count FROM address GROUP BY country;
30
31 SELECT COUNT(*) AS order_with_product, SUM(quantity) AS product_sold FROM order_item WHERE product_id = 1;
32
33 SELECT MAX(order_price) AS max_order_price FROM orders;
34
35 SELECT status, COUNT(*) AS orders_count FROM orders GROUP BY status;
36
37 SELECT client_id, ROUND(AVG(order_price), 2) AS avg_spent FROM orders GROUP BY client_id;
Data Output
Messages
Notifications
SQL
Showing rows: 1 to 1
Page No: 1
of 1
order_with_product bigint
product_sold bigint
1 2 2
Total rows: 1
Query complete 00:00:00.090
CRLF
Ln 32, Col 1

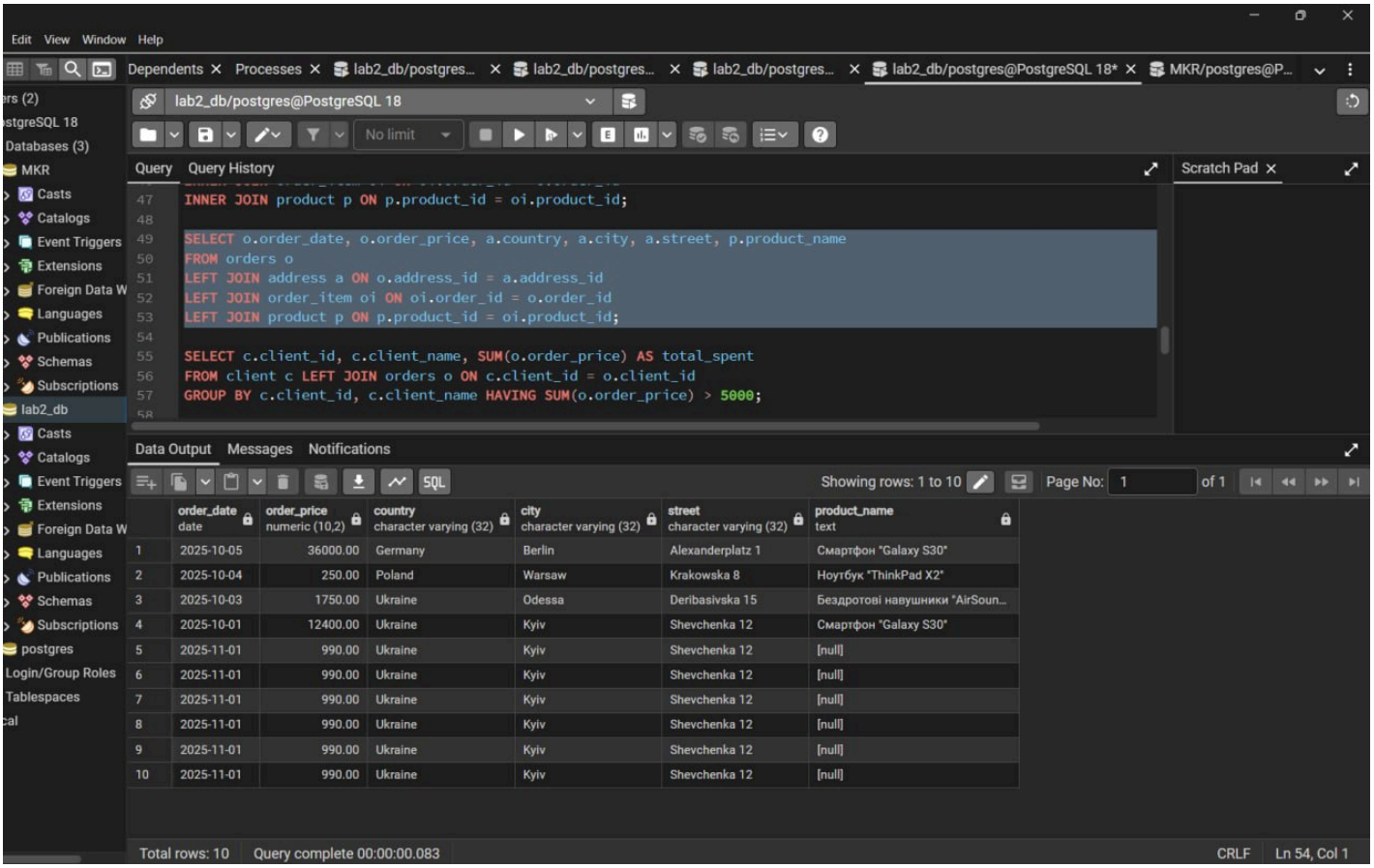

Window Help
Dependents
Processes
lab2_db/postgres...
lab2_db/postgres...
lab2_db/postgres...
lab2_db/postgres@PostgreSQL 18*
MKR/postgres@P...
lab2_db/postgres@PostgreSQL 18
No limit
Query
Query History
Scratch Pad
SELECT COUNT(*) AS total_orders FROM orders;
SELECT country, COUNT(*) AS addresses_count FROM address GROUP BY country;
SELECT COUNT(*) AS order_with_product, SUM(quantity) AS product_sold FROM order_item WHERE product_id = 1;
SELECT MAX(order_price) AS max_order_price FROM orders;
SELECT status, COUNT(*) AS orders_count FROM orders GROUP BY status;
SELECT client_id, ROUND(AVG(order_price), 2) AS avg_spent FROM orders GROUP BY client_id;
Data Output
Messages
Notifications
SQL
Showing rows: 1 to 1
Page No: 1
of 1
max_order_price
numeric
1
36000.00
Total rows: 1
Query complete 00:00:00.095
CRLF
Ln 33, Col 56

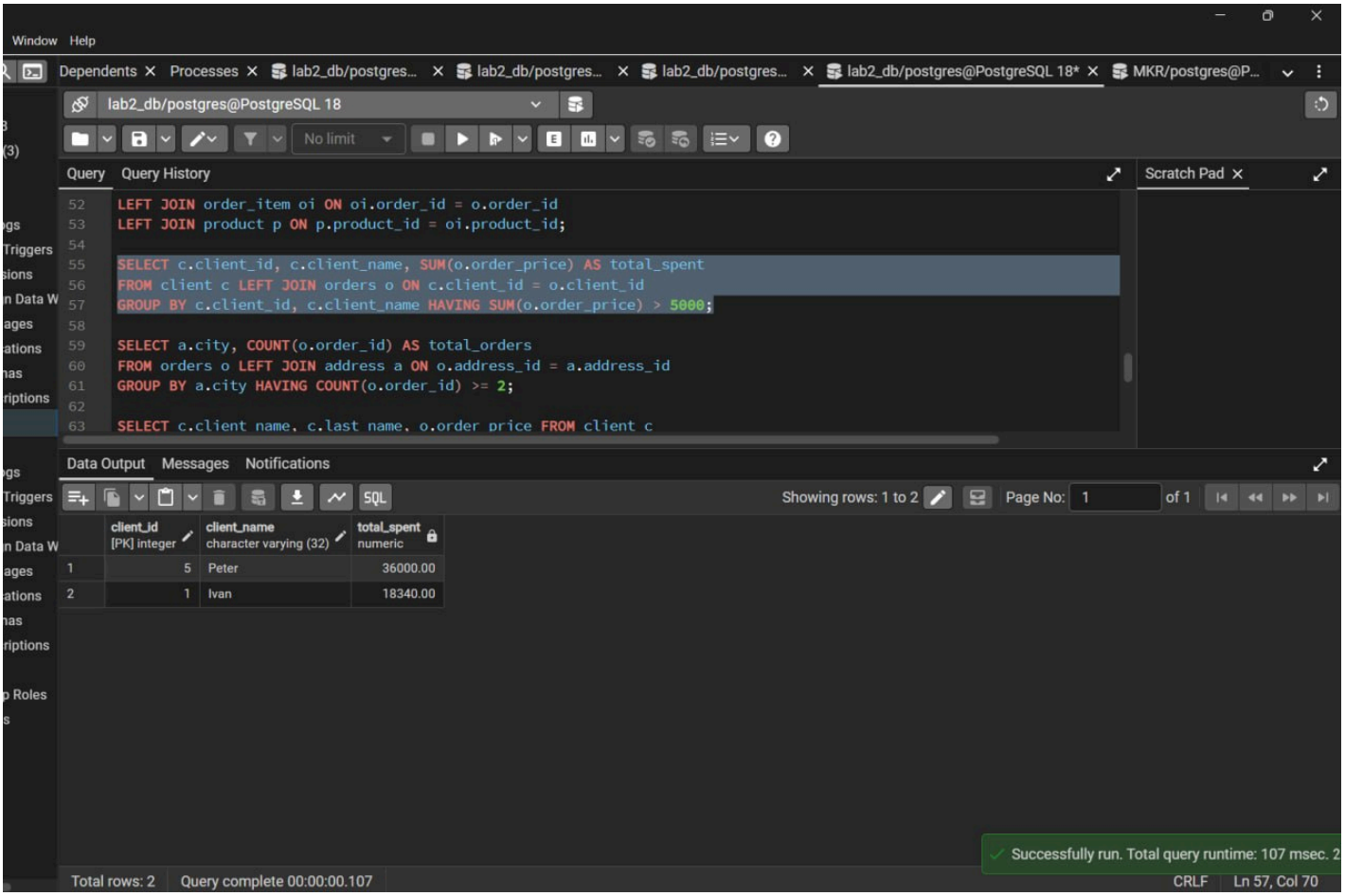

Window Help
Dependents
Processes
lab2_db/postgres...
lab2_db/postgres...
lab2_db/postgres...
lab2_db/postgres@PostgreSQL 18*
MKR/postgres@P...
lab2_db/postgres@PostgreSQL 18
No limit
Query
Query History
Scratch Pad
SELECT MAX(order_price) AS max_order_price FROM orders;
SELECT status, COUNT(*) AS orders_count FROM orders GROUP BY status;
SELECT client_id, ROUND(AVG(order_price), 2) AS avg_spent FROM orders GROUP BY client_id;
SELECT payment_status, COUNT(*) AS payments_count FROM payment WHERE price > 1000 GROUP BY payment_status;
SELECT client_id, COUNT(*) AS total_orders, SUM(order_price) AS total_spent FROM orders GROUP BY client_id;
Data Output
Messages
Notifications
SQL
Showing rows: 1 to 3
Page No: 1
of 1
status
status_name
orders_count
bigint
1
confirmed
8
2
delivered
1
3
shipped
1
Successfully run. Total query runtime: 123 msec. 3 rows affected.
Total rows: 3
Query complete 00:00:00.123
CRLF
Ln 35, Col 69

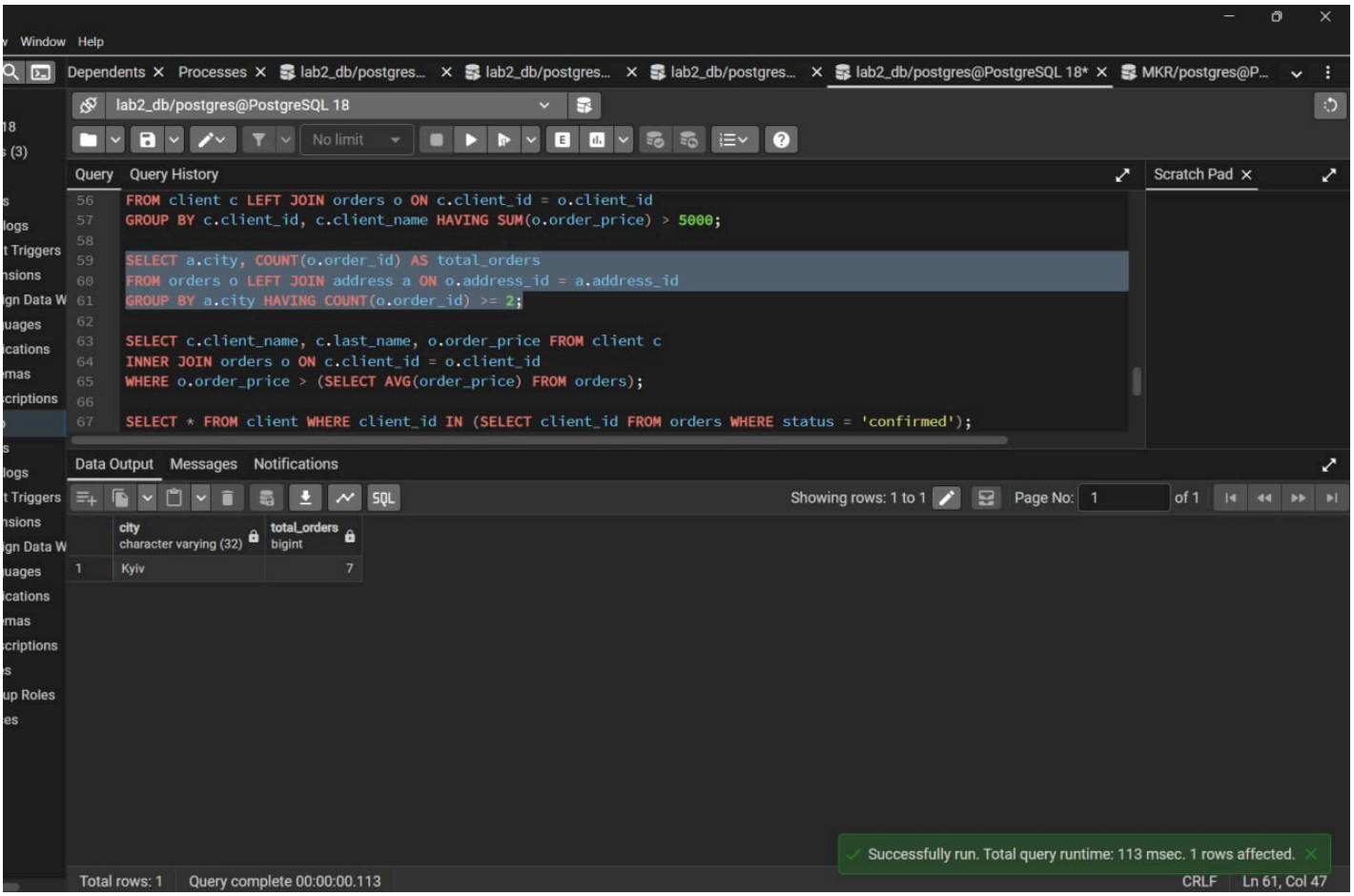

Edit Window Help
Dependents
Processes
lab2_db/postgres...
lab2_db/postgres...
lab2_db/postgres...
lab2_db/postgres@PostgreSQL 18*
MKR/postgres@P...
lab2_db/postgres@PostgreSQL 18
No limit
Query
Query History
Scratch Pad
SELECT MAX(order_price) AS max_order_price FROM orders;
SELECT status, COUNT(*) AS orders_count FROM orders GROUP BY status;
SELECT client_id, ROUND(AVG(order_price), 2) AS avg_spent FROM orders GROUP BY client_id;
SELECT payment_status, COUNT(*) AS payments_count FROM payment WHERE price > 1000 GROUP BY payment_status;
SELECT client_id, COUNT(*) AS total_orders, SUM(order_price) AS total_spent FROM orders GROUP BY client_id;
Data Output
Messages
Notifications
SQL
Showing rows: 1 to 4
Page No: 1
of 1
Edit range
client_id integer
avg_spent numeric
1 3 1750.00
2 5 36000.00
3 4 250.00
4 1 2620.00
Successfully run. Total query runtime: 112 msec. 4 rows affected.
Total rows: 4
Query complete 00:00:00.112
CRLF
Ln 37, Col 90

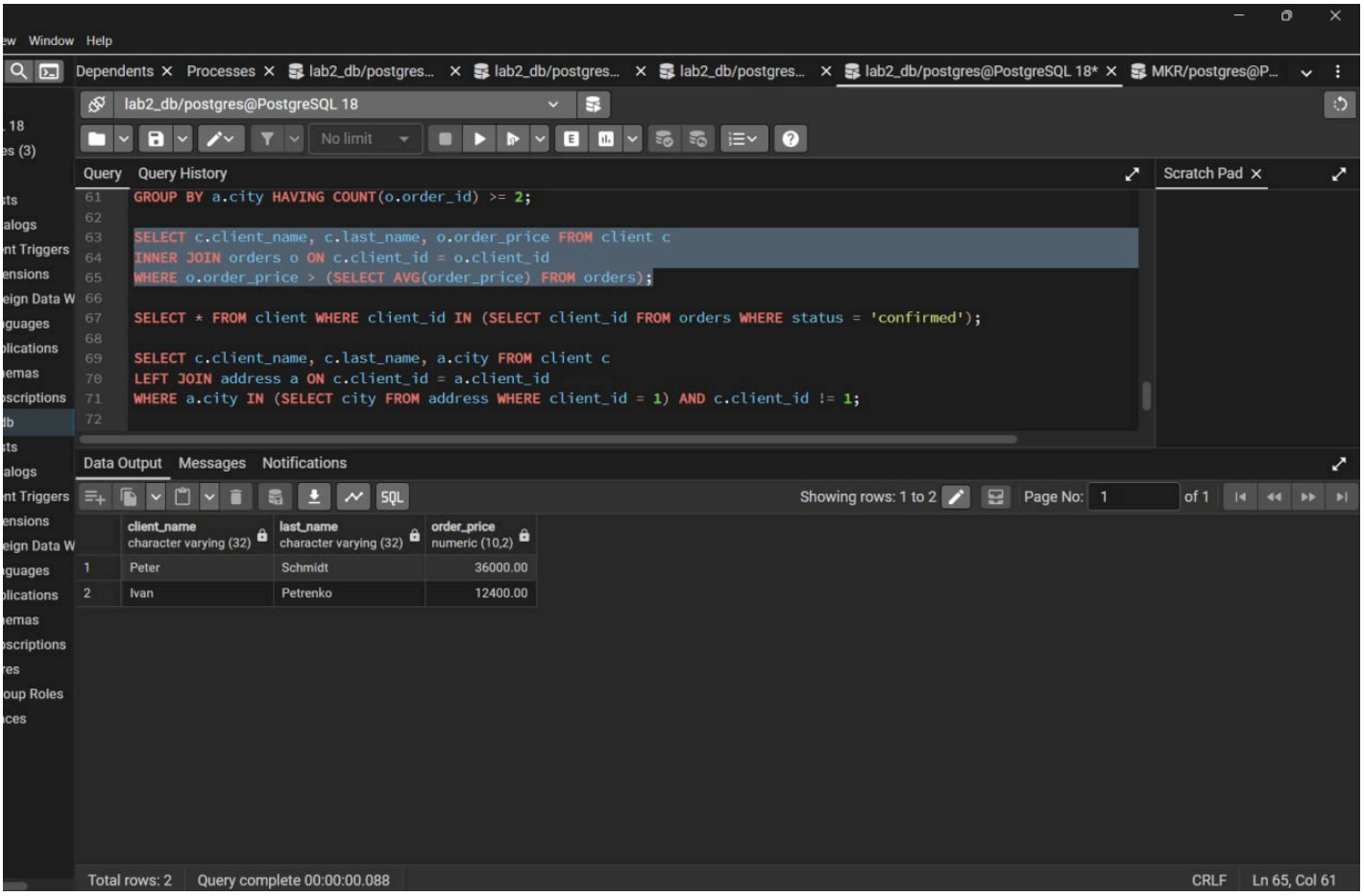

View Window Help
Dependents
Processes
lab2_db/postgres...
lab2_db/postgres...
lab2_db/postgres...
lab2_db/postgres@PostgreSQL 18*
MKR/postgres@P...
lab2_db/postgres@PostgreSQL 18
No limit
Query
Query History
Scratch Pad
SELECT MAX(order_price) AS max_order_price FROM orders;
SELECT status, COUNT(*) AS orders_count FROM orders GROUP BY status;
SELECT client_id, ROUND(AVG(order_price), 2) AS avg_spent FROM orders GROUP BY client_id;
SELECT payment_status, COUNT(*) AS payments_count FROM payment WHERE price > 1000 GROUP BY payment_status;
SELECT client_id, COUNT(*) AS total_orders, SUM(order_price) AS total_spent FROM orders GROUP BY client_id;
Data Output
Messages
Notifications
SQL
Showing rows: 1 to 1
Page No: 1
of 1
payment_status payment_status_name
payments_count bigint
1 paid 3
Total rows: 1
Query complete 00:00:00.103
CRLF
Ln 39, Col 107

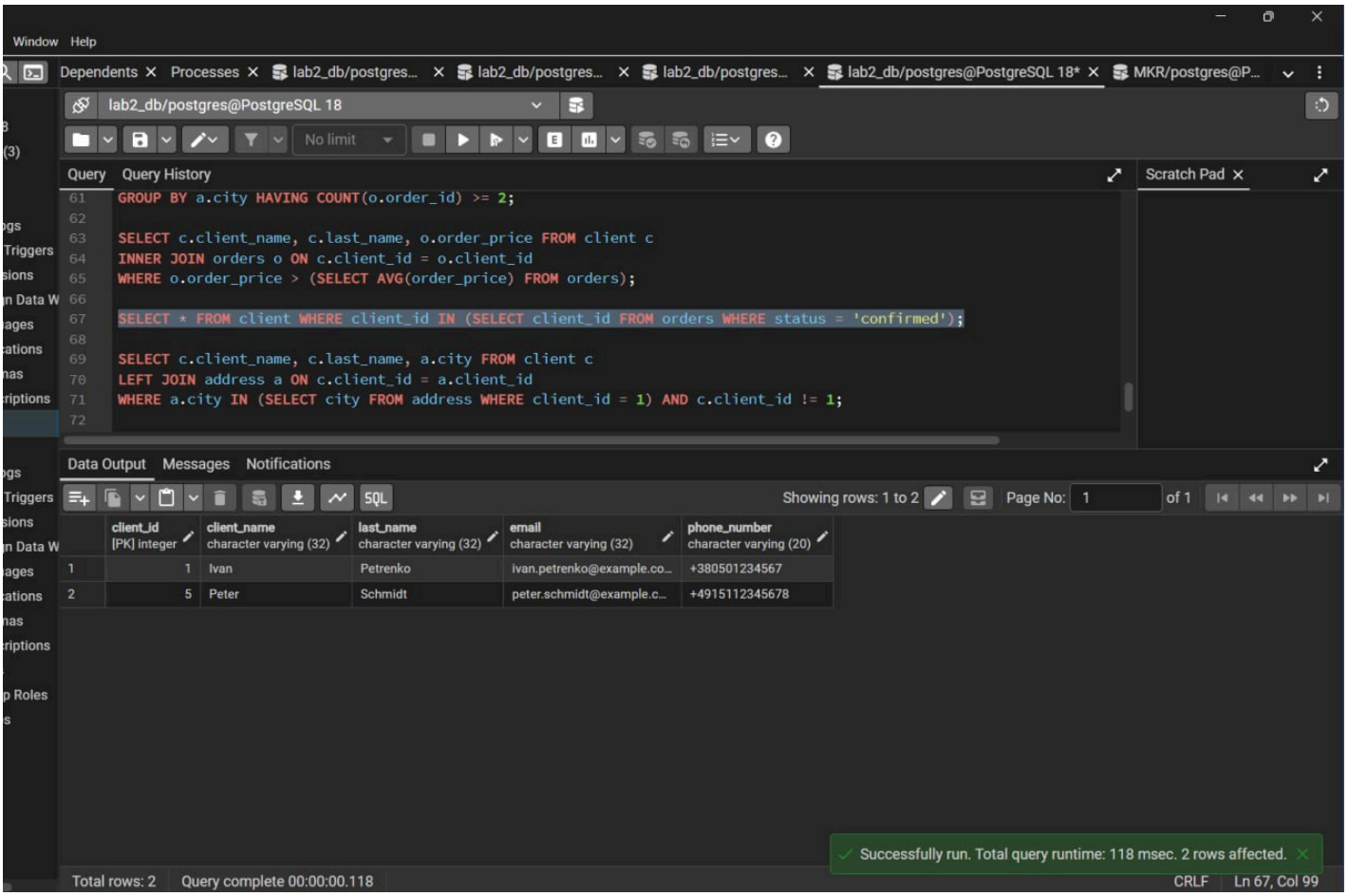

Window Help
Dependents Processes lab2_db/postgres... lab2_db/postgres... lab2_db/postgres... lab2_db/postgres@PostgreSQL 18* MKR/postgres@P...
lab2_db/postgres@PostgreSQL 18
No limit
Query Query History
Scratch Pad
SELECT client_id, ROUND(AVG(order_price), 2) AS avg_spent FROM orders GROUP BY client_id;
SELECT payment_status, COUNT(*) AS payments_count FROM payment WHERE price > 1000 GROUP BY payment_status;
SELECT client_id, COUNT(*) AS total_orders, SUM(order_price) AS total_spent FROM orders GROUP BY client_id;
SELECT o.order_date, o.order_price, c.client_name ' ' c.last_name AS full_name, c.email, c.phone_number, p.product
FROM orders o
INNER JOIN client c ON c.client_id = o.client_id
INNER JOIN order_item oi ON oi.order_id = o.order_id
INNER JOIN product p ON p.product_id = oi.product_id;
Data Output Messages Notifications
Showing rows: 1 to 4 Page No: 1 of 1
client_id integer
total_orders bigint
total_spent numeric
1 3 1 1750.00
2 5 1 36000.00
3 4 1 250.00
4 1 7 18340.00
Total rows: 4 Query complete 00:00:00.105 CRLF Ln 41, Col 108

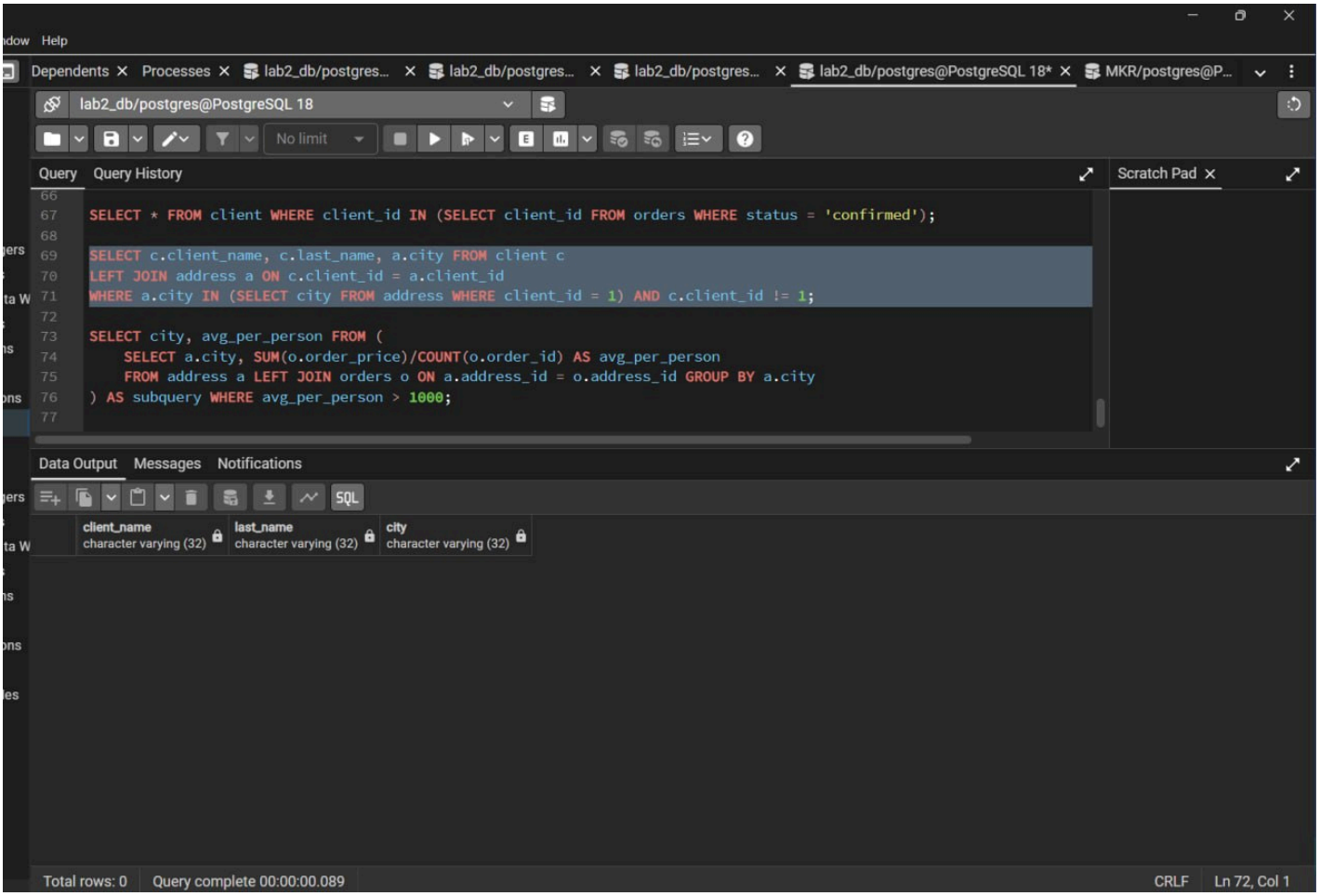

Edit View Window Help
Dependents Processes lab2_db/postgres... lab2_db/postgres... lab2_db/postgres... lab2_db/postgres@PostgreSQL 18* MKR/postgres@P...
lab2_db/postgres@PostgreSQL 18
No limit
Query Query History
Scratch Pad
INNER JOIN product p ON p.product_id = oi.product_id;
SELECT o.order_date, o.order_price, a.country, a.city, a.street, p.product_name
FROM orders o
LEFT JOIN address a ON o.address_id = a.address_id
LEFT JOIN order_item oi ON oi.order_id = o.order_id
LEFT JOIN product p ON p.product_id = oi.product_id;
SELECT c.client_id, c.client_name, SUM(o.order_price) AS total_spent
FROM client c LEFT JOIN orders o ON c.client_id = o.client_id
GROUP BY c.client_id, c.client_name HAVING SUM(o.order_price) > 5000;
Data Output Messages Notifications
Showing rows: 1 to 10 Page No: 1 of 1
order_date date
order_price numeric (10,2)
country character varying (32)
city character varying (32)
street character varying (32)
product_name text
1 2025-10-05 36000.00 Germany Berlin Alexanderplatz 1 Смартфон "Galaxy S30"
2 2025-10-04 250.00 Poland Warsaw Krakowska 8 Ноутбук "ThinkPad X2"
3 2025-10-03 1750.00 Ukraine Odessa Deribasivska 15 Бездротові навушники "AirSoun...
4 2025-10-01 12400.00 Ukraine Kyiv Shevchenka 12 Смартфон "Galaxy S30"
5 2025-11-01 990.00 Ukraine Kyiv Shevchenka 12 [null]
6 2025-11-01 990.00 Ukraine Kyiv Shevchenka 12 [null]
7 2025-11-01 990.00 Ukraine Kyiv Shevchenka 12 [null]
8 2025-11-01 990.00 Ukraine Kyiv Shevchenka 12 [null]
9 2025-11-01 990.00 Ukraine Kyiv Shevchenka 12 [null]
10 2025-11-01 990.00 Ukraine Kyiv Shevchenka 12 [null]
Total rows: 10 Query complete 00:00:00.083 CRLF Ln 54, Col 1


Window Help
Dependents Processes lab2_db/postgres... lab2_db/postgres... lab2_db/postgres... lab2_db/postgres@PostgreSQL 18* MKR/postgres@P...
lab2_db/postgres@PostgreSQL 18
No limit
Query Query History
Scratch Pad
52 LEFT JOIN order_item oi ON oi.order_id = o.order_id
53 LEFT JOIN product p ON p.product_id = oi.product_id;
54
55 SELECT c.client_id, c.client_name, SUM(o.order_price) AS total_spent
56 FROM client c LEFT JOIN orders o ON c.client_id = o.client_id
57 GROUP BY c.client_id, c.client_name HAVING SUM(o.order_price) > 5000;
58
59 SELECT a.city, COUNT(o.order_id) AS total_orders
60 FROM orders o LEFT JOIN address a ON o.address_id = a.address_id
61 GROUP BY a.city HAVING COUNT(o.order_id) >= 2;
62
63 SELECT c.client_name, c.last_name, o.order_price FROM client c
Data Output Messages Notifications
SQL
Showing rows: 1 to 2 Page No: 1 of 1
client_id [PK] integer
client_name character varying (32)
total_spent numeric
1 5 Peter 36000.00
2 1 Ivan 18340.00
Successfully run. Total query runtime: 107 msec. 2
Total rows: 2 Query complete 00:00:00.107
CRLF Ln 57, Col 70


Window Help
Dependents Processes lab2_db/postgres... lab2_db/postgres... lab2_db/postgres... lab2_db/postgres@PostgreSQL 18* MKR/postgres@P...
lab2_db/postgres@PostgreSQL 18
No limit
Query Query History
Scratch Pad
56 FROM client c LEFT JOIN orders o ON c.client_id = o.client_id
57 GROUP BY c.client_id, c.client_name HAVING SUM(o.order_price) > 5000;
58
59 SELECT a.city, COUNT(o.order_id) AS total_orders
60 FROM orders o LEFT JOIN address a ON o.address_id = a.address_id
61 GROUP BY a.city HAVING COUNT(o.order_id) >= 2;
62
63 SELECT c.client_name, c.last_name, o.order_price FROM client c
64 INNER JOIN orders o ON c.client_id = o.client_id
65 WHERE o.order_price > (SELECT AVG(order_price) FROM orders);
66
67 SELECT * FROM client WHERE client_id IN (SELECT client_id FROM orders WHERE status = 'confirmed');
Data Output Messages Notifications
SQL
Showing rows: 1 to 1 Page No: 1 of 1
city character varying (32)
total_orders bigint
1 Kyiv 7
Successfully run. Total query runtime: 113 msec. 1 rows affected.
Total rows: 1 Query complete 00:00:00.113
CRLF Ln 61, Col 47


Window Help
Dependents × Processes × lab2_db/postgres... × lab2_db/postgres... × lab2_db/postgres... × lab2_db/postgres@PostgreSQL 18* × MKR/postgres@P...
lab2_db/postgres@PostgreSQL 18
No limit
Query Query History
Scratch Pad ×
61 GROUP BY a.city HAVING COUNT(o.order_id) >= 2;
62
63 SELECT c.client_name, c.last_name, o.order_price FROM client c
64 INNER JOIN orders o ON c.client_id = o.client_id
65 WHERE o.order_price > (SELECT AVG(order_price) FROM orders);
66
67 SELECT * FROM client WHERE client_id IN (SELECT client_id FROM orders WHERE status = 'confirmed');
68
69 SELECT c.client_name, c.last_name, a.city FROM client c
70 LEFT JOIN address a ON c.client_id = a.client_id
71 WHERE a.city IN (SELECT city FROM address WHERE client_id = 1) AND c.client_id != 1;
72
Data Output Messages Notifications
SQL
Showing rows: 1 to 2 Page No: 1 of 1
client_name character varying (32) | last_name character varying (32) | order_price numeric (10,2)
1 Peter Schmidt 36000.00
2 Ivan Petrenko 12400.00
Total rows: 2 Query complete 00:00:00.088 CRLF Ln 65, Col 61


Window Help
Dependents × Processes × lab2_db/postgres... × lab2_db/postgres... × lab2_db/postgres... × lab2_db/postgres@PostgreSQL 18* × MKR/postgres@P...
lab2_db/postgres@PostgreSQL 18
No limit
Query Query History
Scratch Pad ×
61 GROUP BY a.city HAVING COUNT(o.order_id) >= 2;
62
63 SELECT c.client_name, c.last_name, o.order_price FROM client c
64 INNER JOIN orders o ON c.client_id = o.client_id
65 WHERE o.order_price > (SELECT AVG(order_price) FROM orders);
66
67 SELECT * FROM client WHERE client_id IN (SELECT client_id FROM orders WHERE status = 'confirmed');
68
69 SELECT c.client_name, c.last_name, a.city FROM client c
70 LEFT JOIN address a ON c.client_id = a.client_id
71 WHERE a.city IN (SELECT city FROM address WHERE client_id = 1) AND c.client_id != 1;
72
Data Output Messages Notifications
SQL
Showing rows: 1 to 2 Page No: 1 of 1
client_id [PK] integer | client_name character varying (32) | last_name character varying (32) | email character varying (32) | phone_number character varying (20)
1 1 Ivan Petrenko ivan.petrenko@example.co... +380501234567
2 5 Peter Schmidt peter.schmidt@example.c... +4915112345678
Successfully run. Total query runtime: 118 msec. 2 rows affected.
Total rows: 2 Query complete 00:00:00.118 CRLF Ln 67, Col 99



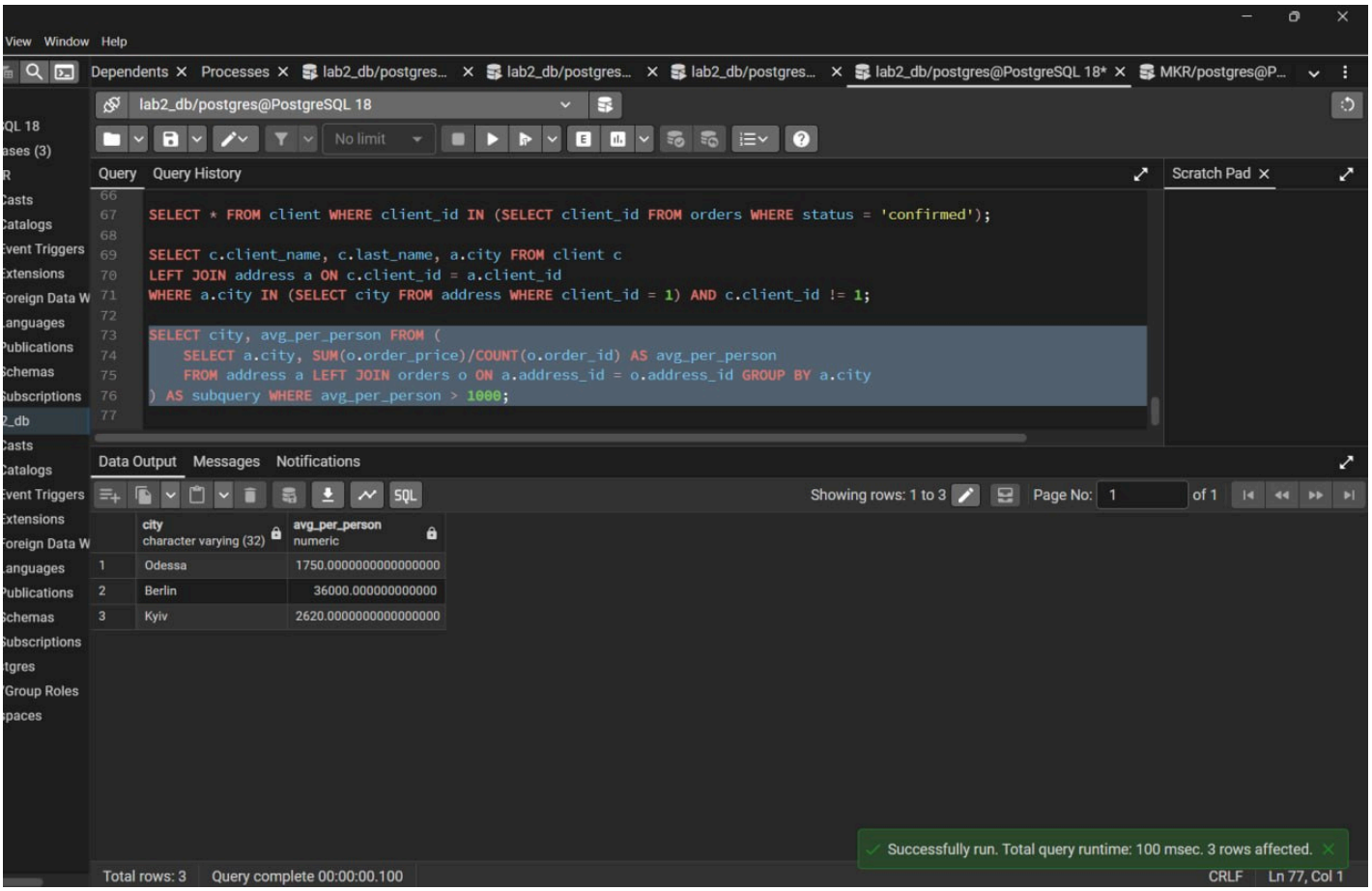
```
SELECT * FROM client WHERE client_id IN (SELECT client_id FROM orders WHERE status = 'confirmed');

SELECT c.client_name, c.last_name, a.city FROM client c
LEFT JOIN address a ON c.client_id = a.client_id
WHERE a.city IN (SELECT city FROM address WHERE client_id = 1) AND c.client_id != 1;

SELECT city, avg_per_person FROM (
    SELECT a.city, SUM(o.order_price)/COUNT(o.order_id) AS avg_per_person
    FROM address a LEFT JOIN orders o ON a.address_id = o.address_id GROUP BY a.city
) AS subquery WHERE avg_per_person > 1000;
```

client_name character varying (32)	last_name character varying (32)	city character varying (32)

Total rows: 0 | Query complete 00:00:00.089 | CRLF | Ln 72, Col 1



```
SELECT * FROM client WHERE client_id IN (SELECT client_id FROM orders WHERE status = 'confirmed');

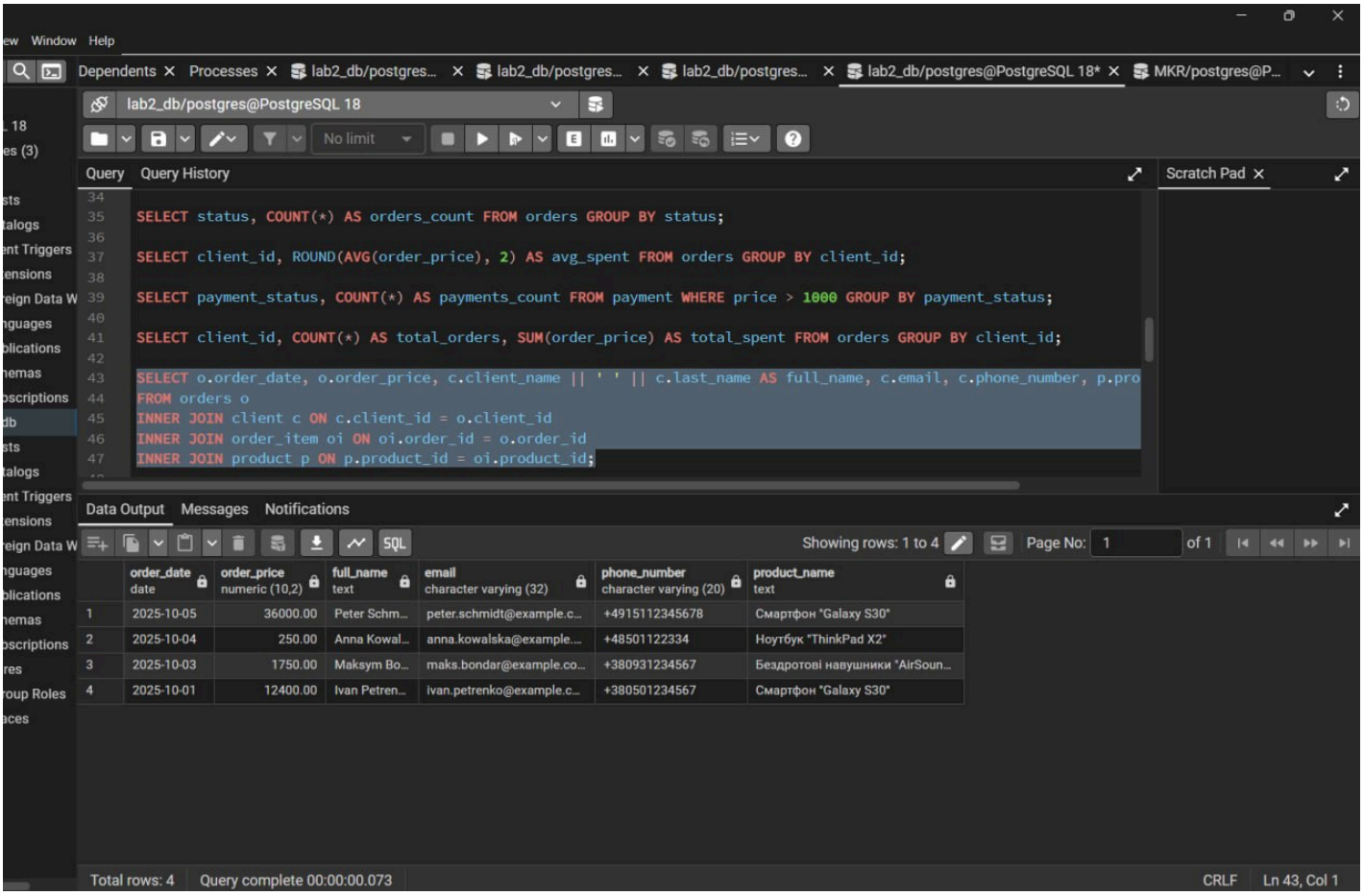
SELECT c.client_name, c.last_name, a.city FROM client c
LEFT JOIN address a ON c.client_id = a.client_id
WHERE a.city IN (SELECT city FROM address WHERE client_id = 1) AND c.client_id != 1;

SELECT city, avg_per_person FROM (
    SELECT a.city, SUM(o.order_price)/COUNT(o.order_id) AS avg_per_person
    FROM address a LEFT JOIN orders o ON a.address_id = o.address_id GROUP BY a.city
) AS subquery WHERE avg_per_person > 1000;
```

	city character varying (32)	avg_per_person numeric
1	Odessa	1750.0000000000000000
2	Berlin	36000.000000000000
3	Kyiv	2620.0000000000000000

Successfully run. Total query runtime: 100 msec. 3 rows affected.

Total rows: 3 | Query complete 00:00:00.100 | CRLF | Ln 77, Col 1



Висновок:

Під час виконання лабораторної роботи №4 було успішно досліджено та практично відпрацьовано побудову аналітичних SQL-запитів (OLAP) у СУБД PostgreSQL:

1. Засвоєно роботу агрегатних функцій (`SUM`, `AVG`, `COUNT`, `MIN`, `MAX`) для обчислення узагальнених показників функціонування інтернет-магазину.
2. Опрацьовано використання оператора `GROUP BY` для зведення даних у групи та `HAVING` для їхнього підсумкового фільтрування за аналітичними критеріями.
3. Реалізовано комбіновані операції з'єднання таблиць (`INNER JOIN`, `LEFT JOIN`, `CROSS JOIN`), що дозволило формувати комплексну звітність із поєднанням інформації про клієнтів, замовлення, адреси та працівників.
4. Застосовано підзапити у блоках `WHERE`, `SELECT`, `HAVING` та `FROM` для реалізації багаторівневих аналітичних обчислень. Усі 27 запитів виконуються без помилок і відповідають усім вимогам методичного посібника.